
Introduction to Neural Networks, Backpropagation, and Gradient Descent

Zonghua Gu

Jun 01 2026, Memorial University



Overview

Goals of this lecture:

- Explain what a neural network is and how it works
- Why you should use them, and why not
- Which neural networks are used today
- Where neural networks are headed next

Outline:

- Building a Neural Network from Scratch
- Gradient Descent
- Types of Neural Networks
- Training tips



ML Training paradigms

- **Supervised (focus of this lecture)**
 - Train a model with explicit input-output pairs
- **Unsupervised**
 - Learns “patterns” from unlabelled data
- **Semi-supervised learning**
 - Learn a few things with input-output pairs, relate them to patterns learnt unsupervised
- **Reinforcement Learning**
 - Learn an optimal “policy” that gives you the best action to take at any given state space by taking random actions and learning through positive or negative reinforcement.



Supervised Learning

Given: Input-output examples of the form:

$$S = (\mathbf{x}_i, \mathbf{y}_i)_{i=1, \dots, T} \quad \mathbf{x}_i \in \mathbb{R}^N, \mathbf{y}_i \in \mathbb{R}^M$$

Assumption: Data is generated by a “true ” function, with some added noise:

$$\mathbf{y}_i = f(\mathbf{x}_i) + v_i$$

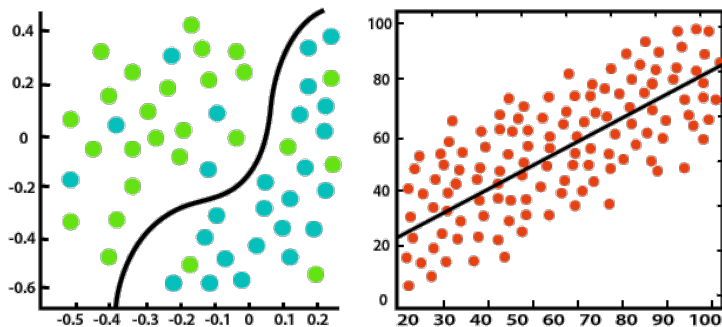
Goal: Learn an approximation $\hat{f}(\mathbf{x})$ of the generator function to use on new data:

$$\hat{f}(\mathbf{x}) \approx f(\mathbf{x})$$

Loss function: A distance between $\hat{f}(\mathbf{x})$ and $f(\mathbf{x})$ such that we can say $\hat{f}(\mathbf{x})$ is “good” if L is low across many given instances of S .

$$L : \mathbb{R}^M \times \mathbb{R}^M \rightarrow \mathbb{R}^{\geq 0}$$

Aim: Learn a function with low “risk”



Classification

Regression

In **classification** (left), the goal is learn a decision boundary that separates different classes.

In **regression** (right), the goal is to predict a continuous numerical value by fitting a function that captures the relationship between input and output.

Risk or Loss: What we want to minimize

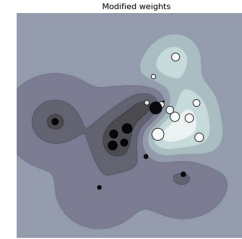
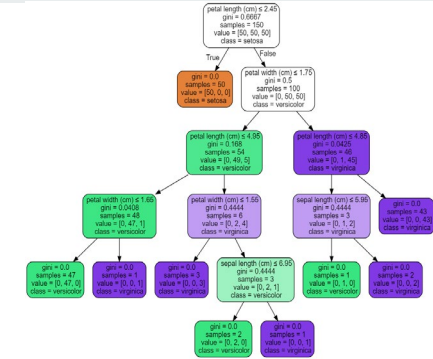
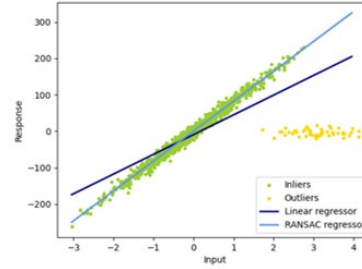
$$R(\hat{f}) = E[L(\hat{f}(X), Y)]$$

Empirical Risk: What we can actually calculate (loss for a candidate model h , averaged over N training examples)

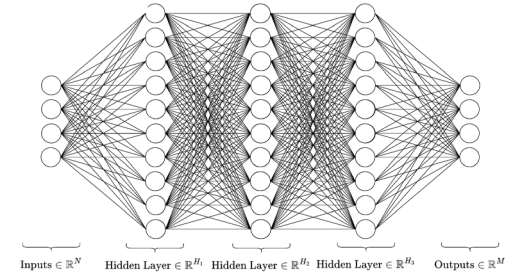
$$R^{\text{emp}}(h) = 1/N \sum_{i=1}^N L(h(\mathbf{x}_i), \mathbf{y}_i)$$

Common Approaches

- Linear/ Polynomial/ Logistic Regression
- (Boosted) Decision trees
- Support Vector Machines
- Naive Bayes
- Neural Networks
- ...



$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$



Artificial vs Biological NNs

ANNs initially inspired by the brain:

Alexander Bain (1873), William James (1890)

Electrical connections/ flow of neurons result in thought and movement

McCulloch & Pitts (1943)

Modern mathematical “artificial” NN models

Rosenblatt (1958)

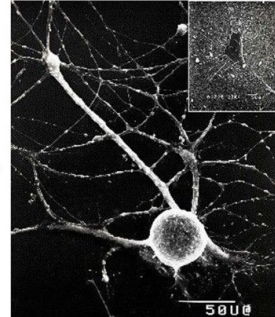
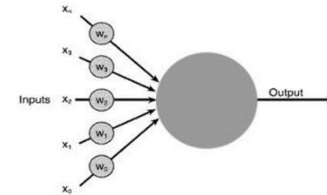
Description of the *perceptron*

Rumelhart, Hinton & Williams (1986)

Multi-layer perceptrons and error backpropagation (learning principle)

Modern:

- ANNs used *everywhere* for *everything*!
- Simplified, abstracted version of biological neurons
- Biologically implausible
- Yann LeCunn calls them “units” instead of “neurons”

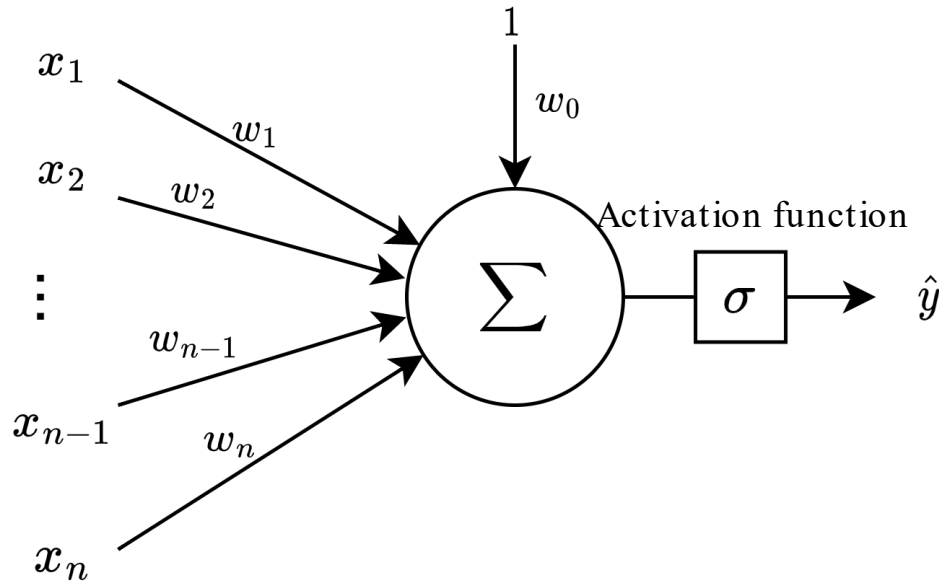


[linkedin.com/company/deeplearningai](https://www.linkedin.com/company/deeplearningai)

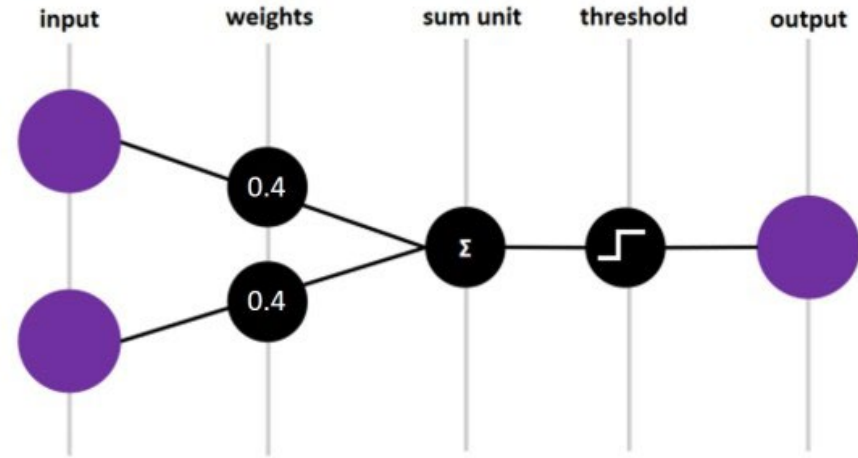
Building a Neural Network From Scratch

Perceptron

$$\hat{y} = \sigma(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$

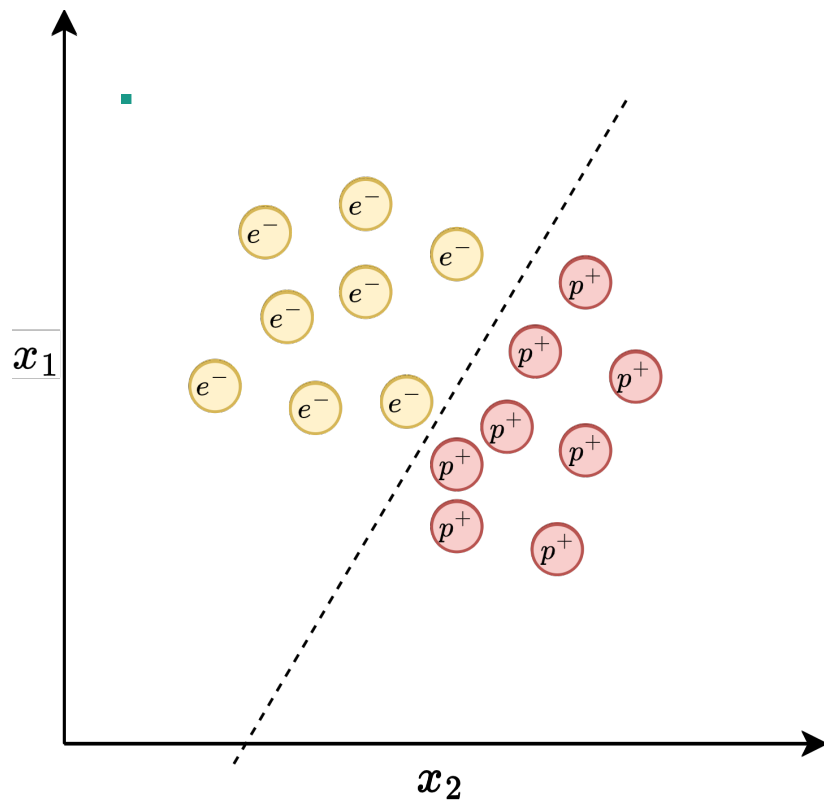


- A single “neuron”
- Only works on **linearly separable** classes

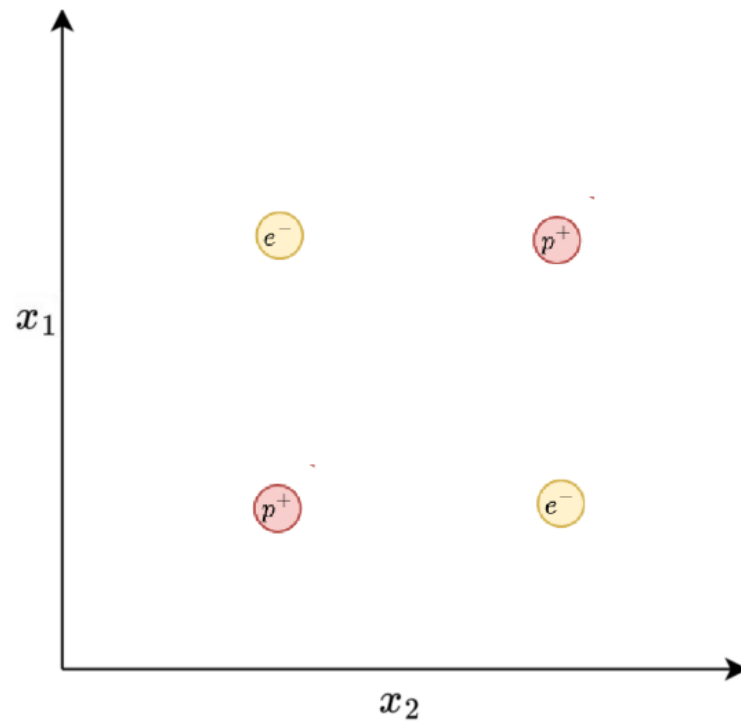


An example of perceptron w/ 2 inputs and binary threshold activation function.

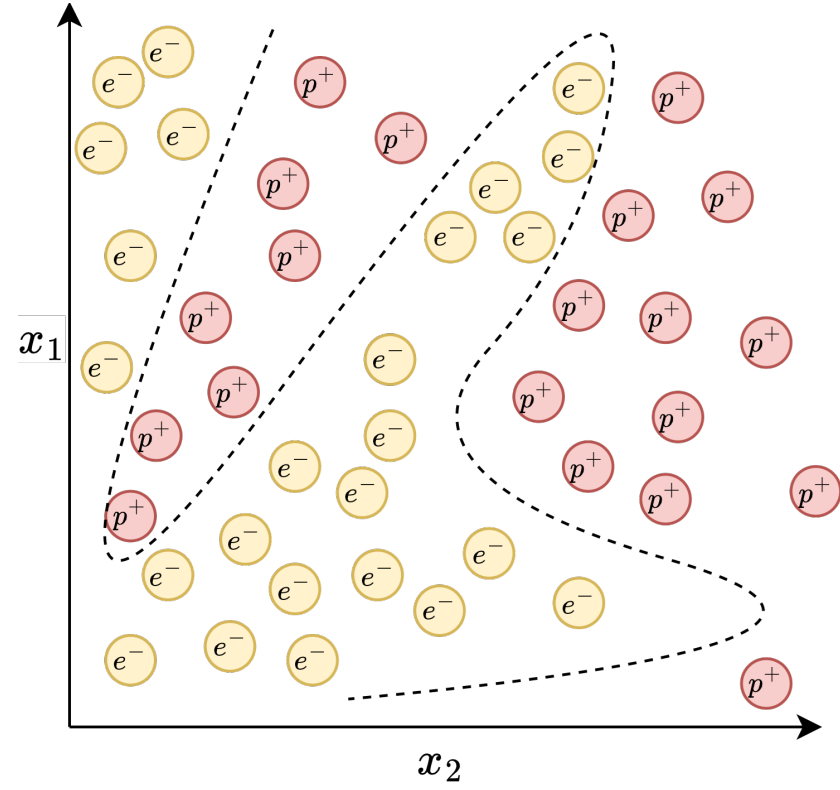
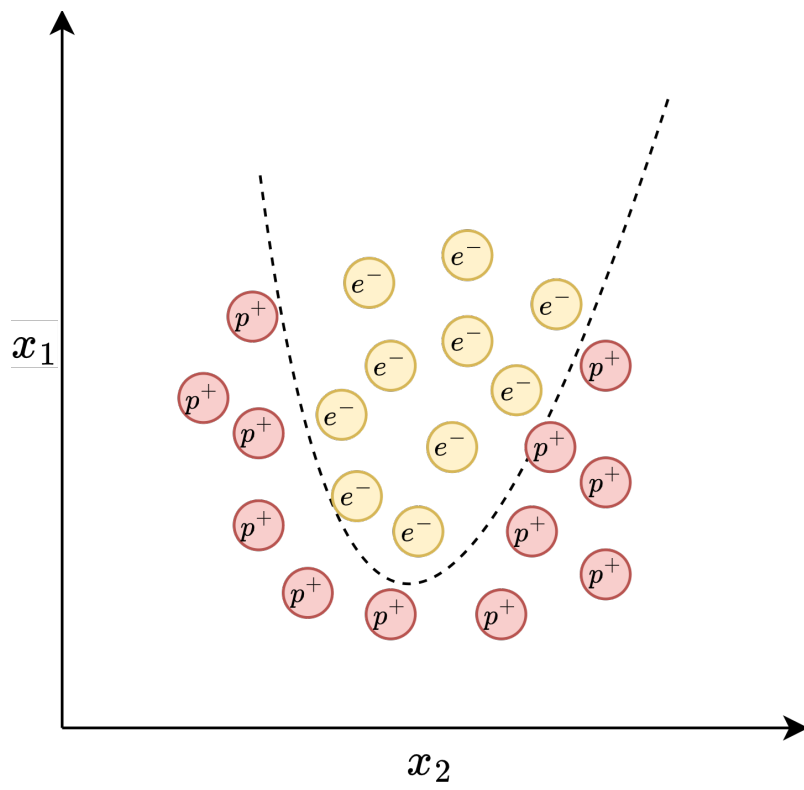
$$\tau(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$



Two classes are *linearly separable*

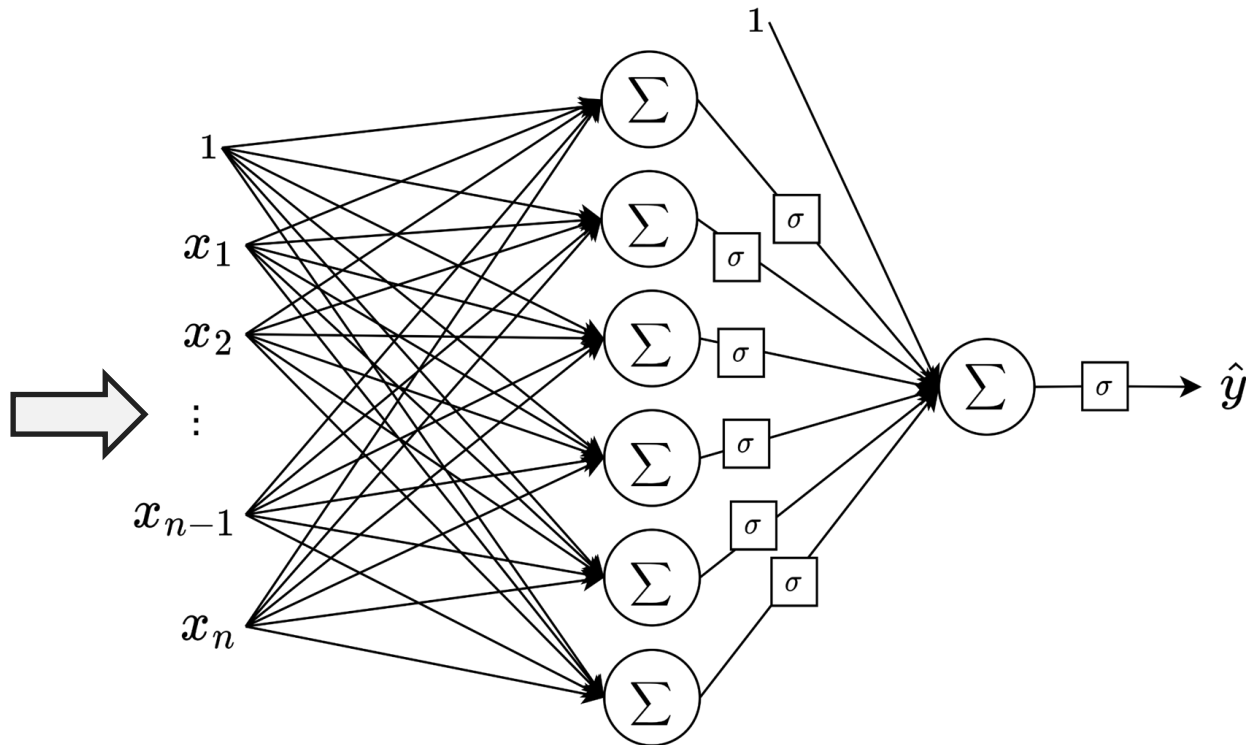
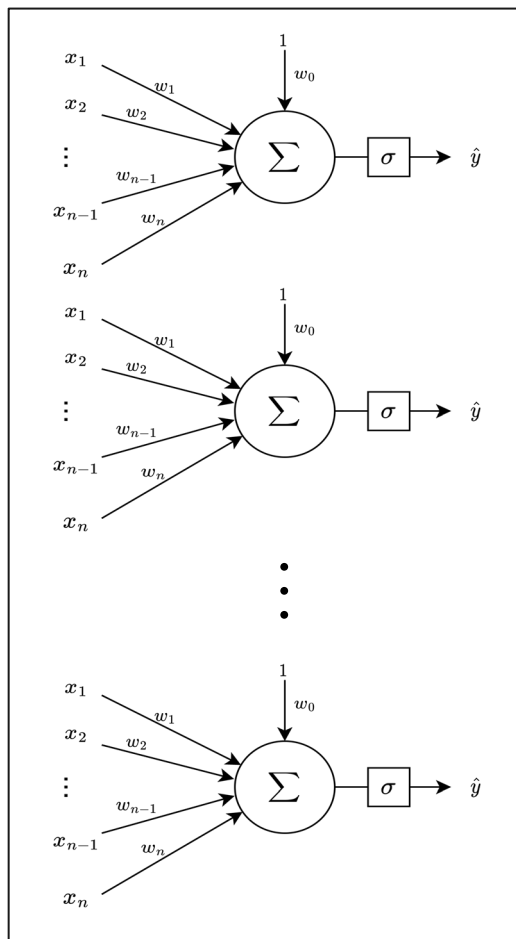


Two classes are *not linearly separable*:
no single straight line can separate them



More cases of non-linearly-separable datasets

Multi-layer Perceptron (MLP)



If we put multiple perceptrons together, with one or more hidden layers, then we can approximate a complex function as a piecewise linear function, making it work for non-linearly-separable datasets.

Activation/
output
of neuron k

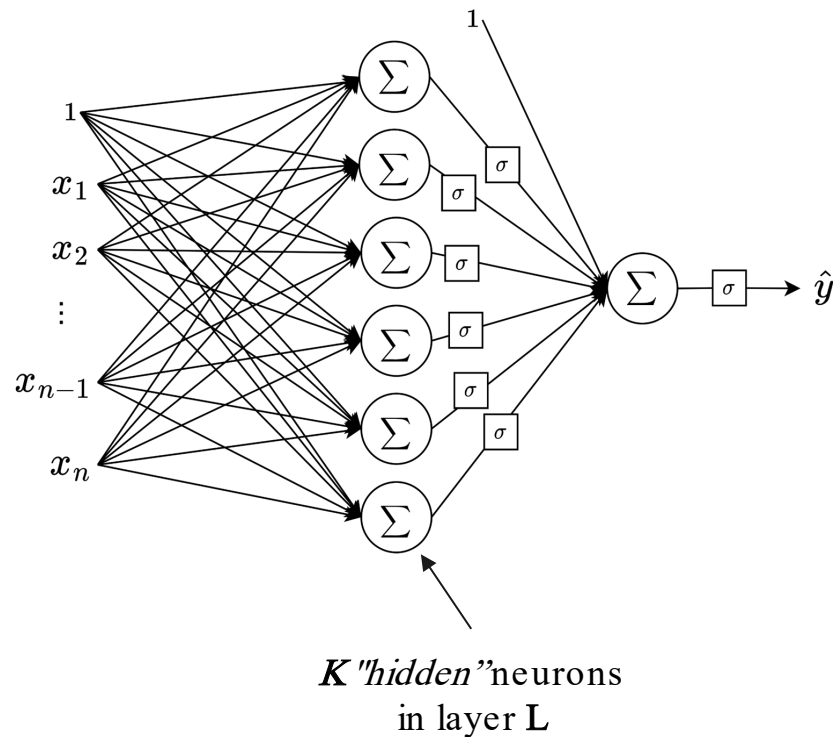


$$o^k = [w_0^k \ w_1^k \ \dots \ w_n^k]$$

↑
“Weight vector”

$$\begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

↑
“Feature vector”



$$\begin{array}{c}
 o^1 = [w_0^1 \ w_1^1 \ \cdots \ w_n^1] \\
 o^2 = [w_0^2 \ w_1^2 \ \cdots \ w_n^2] \\
 \vdots \\
 o^k = [w_0^k \ w_1^k \ \cdots \ w_n^k]
 \end{array}
 \begin{array}{c}
 \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \\
 \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \\
 \vdots \\
 \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}
 \end{array}
 \xrightarrow{\quad}
 o^L =
 \begin{bmatrix}
 w_0^1 & w_1^1 & \cdots & w_n^1 \\
 w_0^2 & w_1^2 & \cdots & w_n^2 \\
 \vdots & \vdots & \cdots & \vdots \\
 w_0^k & w_1^k & \cdots & w_n^k
 \end{bmatrix}
 \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$o^L = W^* \vec{x}^*$

$$o^L = \begin{bmatrix} w_1^1 & w_2^1 & \cdots & w_n^1 \\ w_1^2 & w_2^2 & \cdots & w_n^2 \\ \vdots & \vdots & \ddots & \vdots \\ w_1^k & w_2^k & \cdots & w_n^k \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} w_0^1 \\ w_0^2 \\ \vdots \\ w_0^k \end{bmatrix}$$

Output of each layer is the product of its **weight matrix** and the **input vector** plus its **bias vector**, followed by a **non-linear activation function**.

$$o^L = W\vec{x} + \vec{b}$$

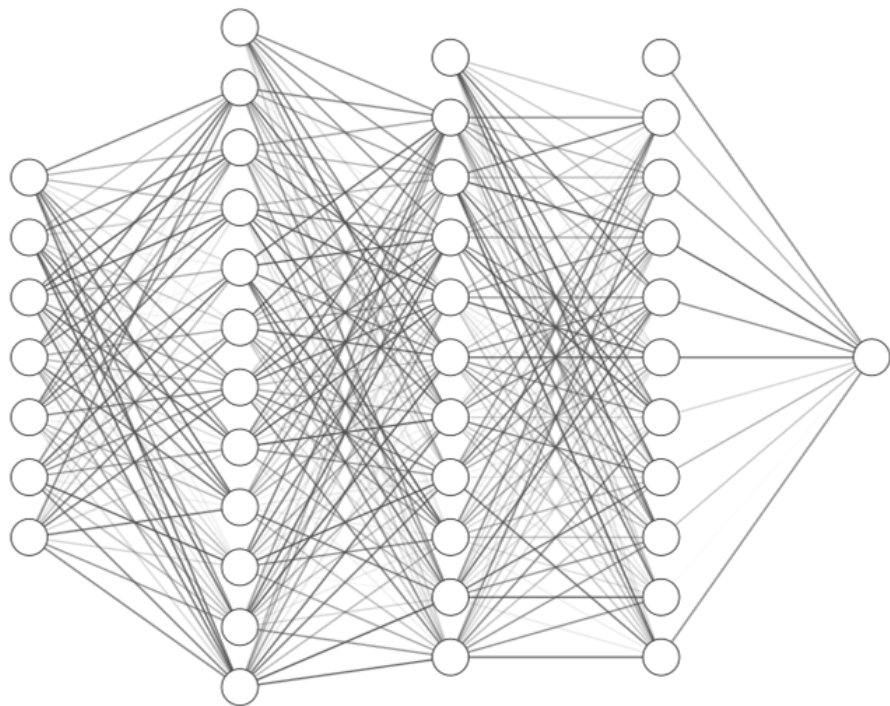
$$\hat{y} = \sigma(o^L)$$

“Activation”

“Bias”

$$\hat{y} = \sigma(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$

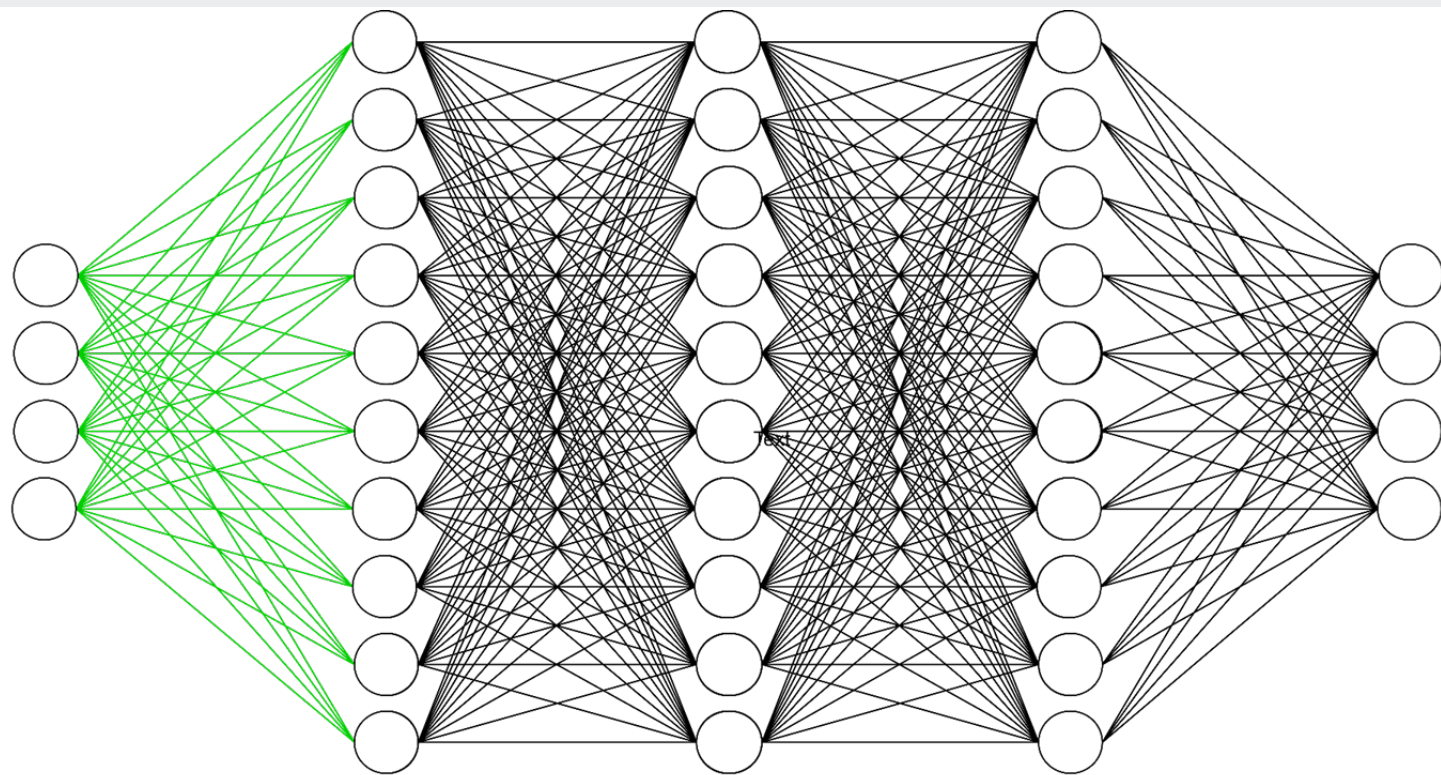
“Activation function”



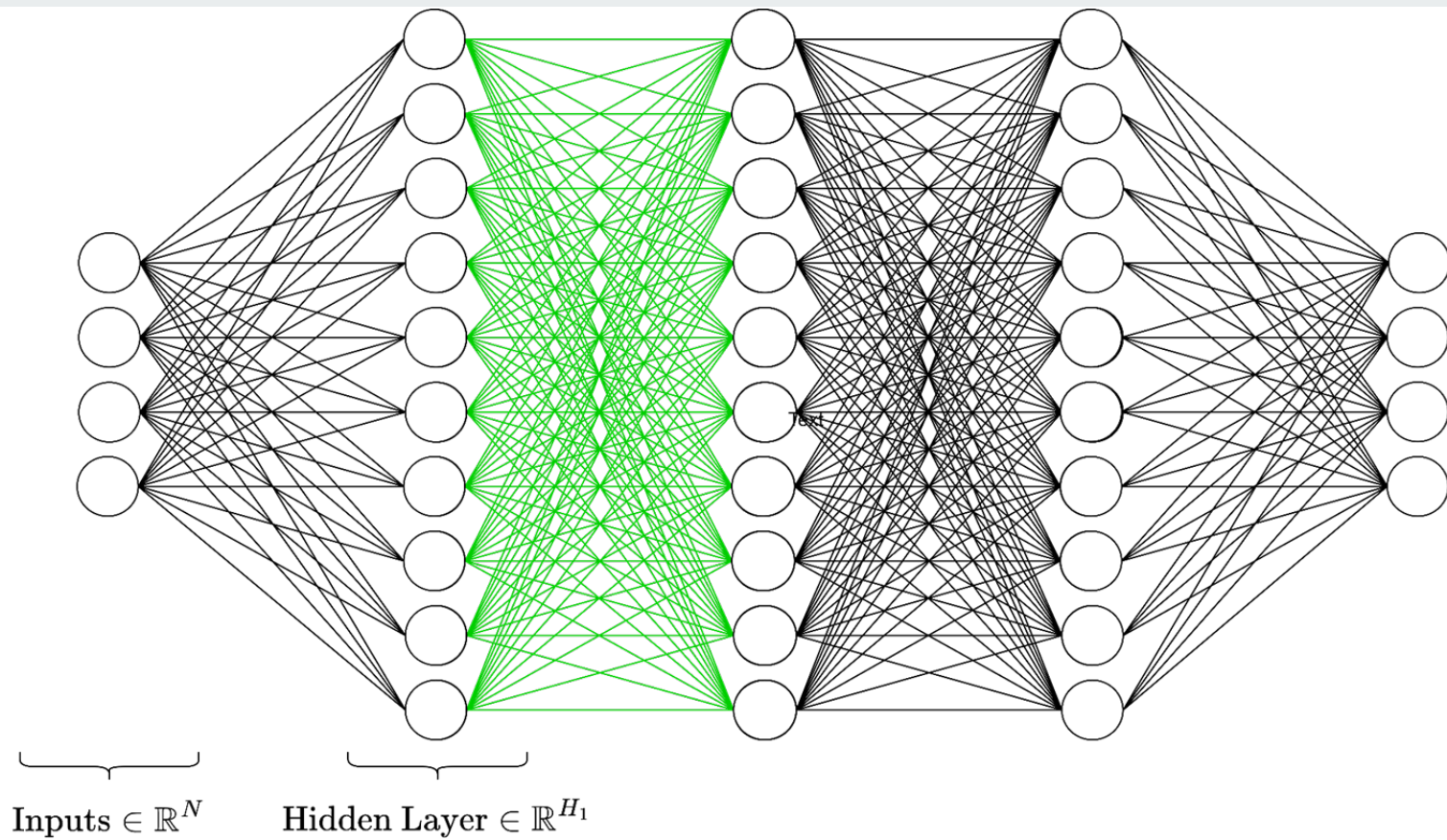
A multi-layer perceptron is a series of affine transformations of an input vector, each of which is wrapped in a **non-linear activation function**.

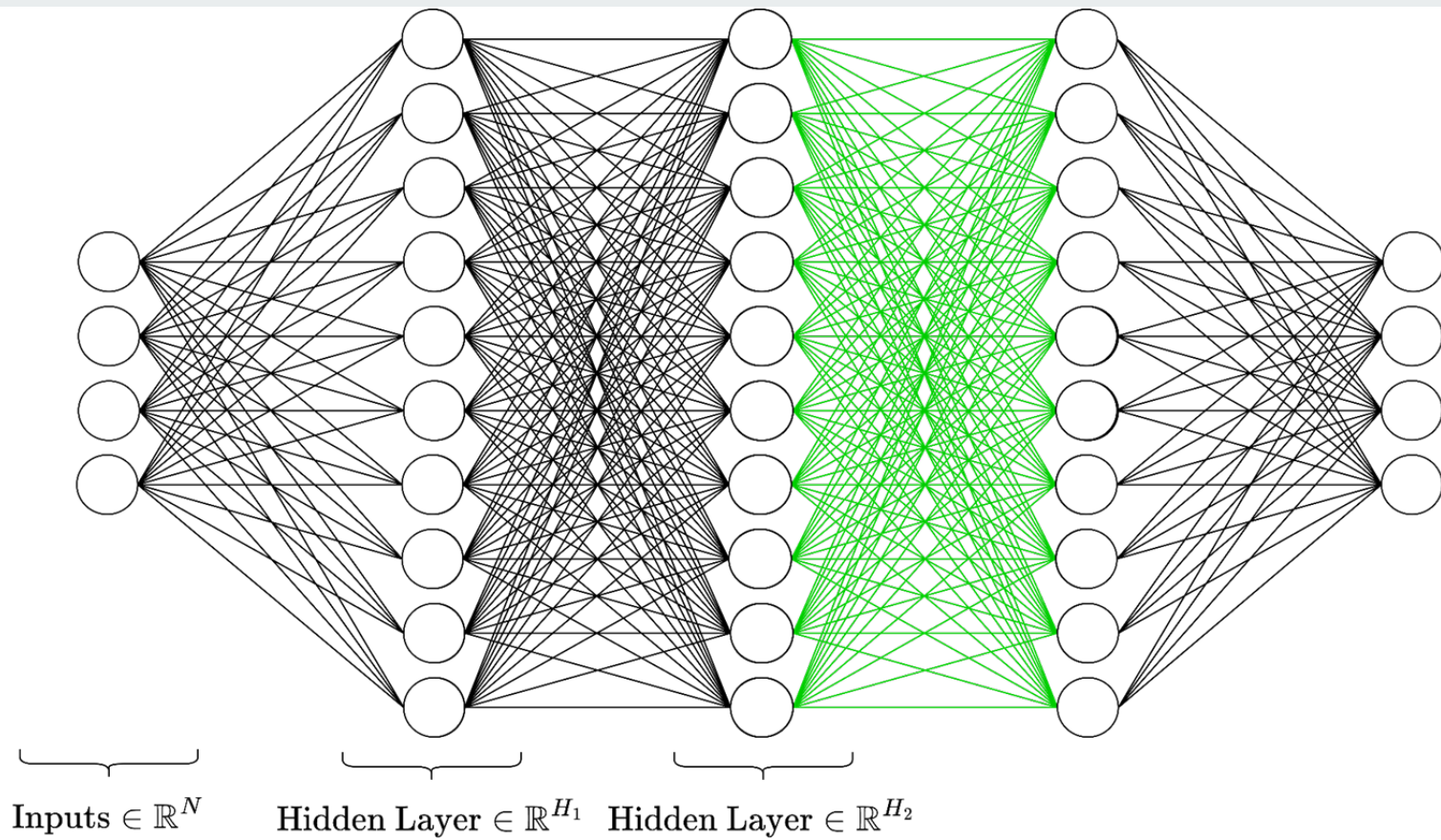
$$\mathcal{N} : \mathbb{R}^N \rightarrow \mathbb{R}^M$$
$$N, M \in \mathbb{N}$$

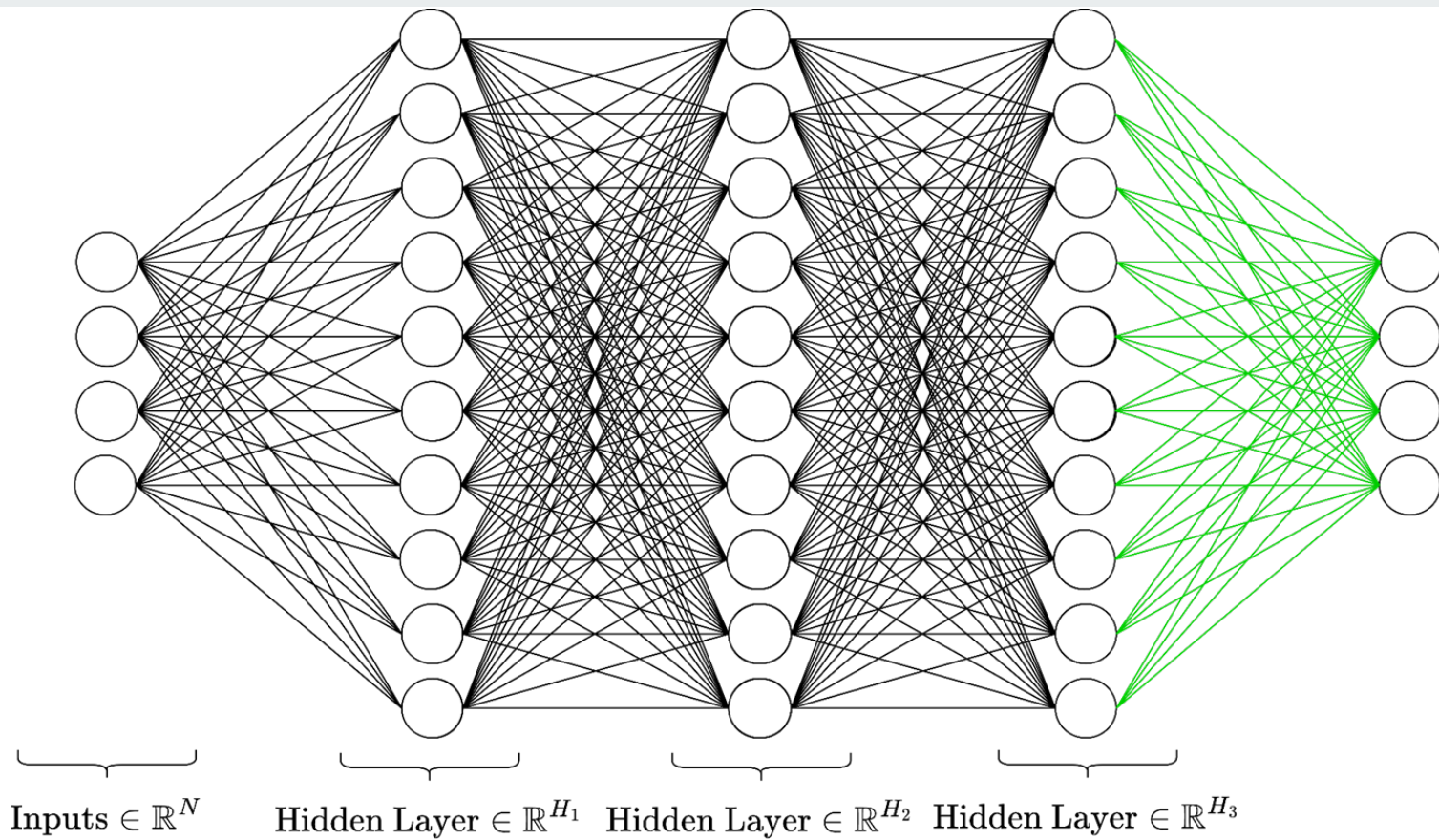
Without a non-linear activation function, a series of linear transformations would result in just a linear transformation of the input to the output.

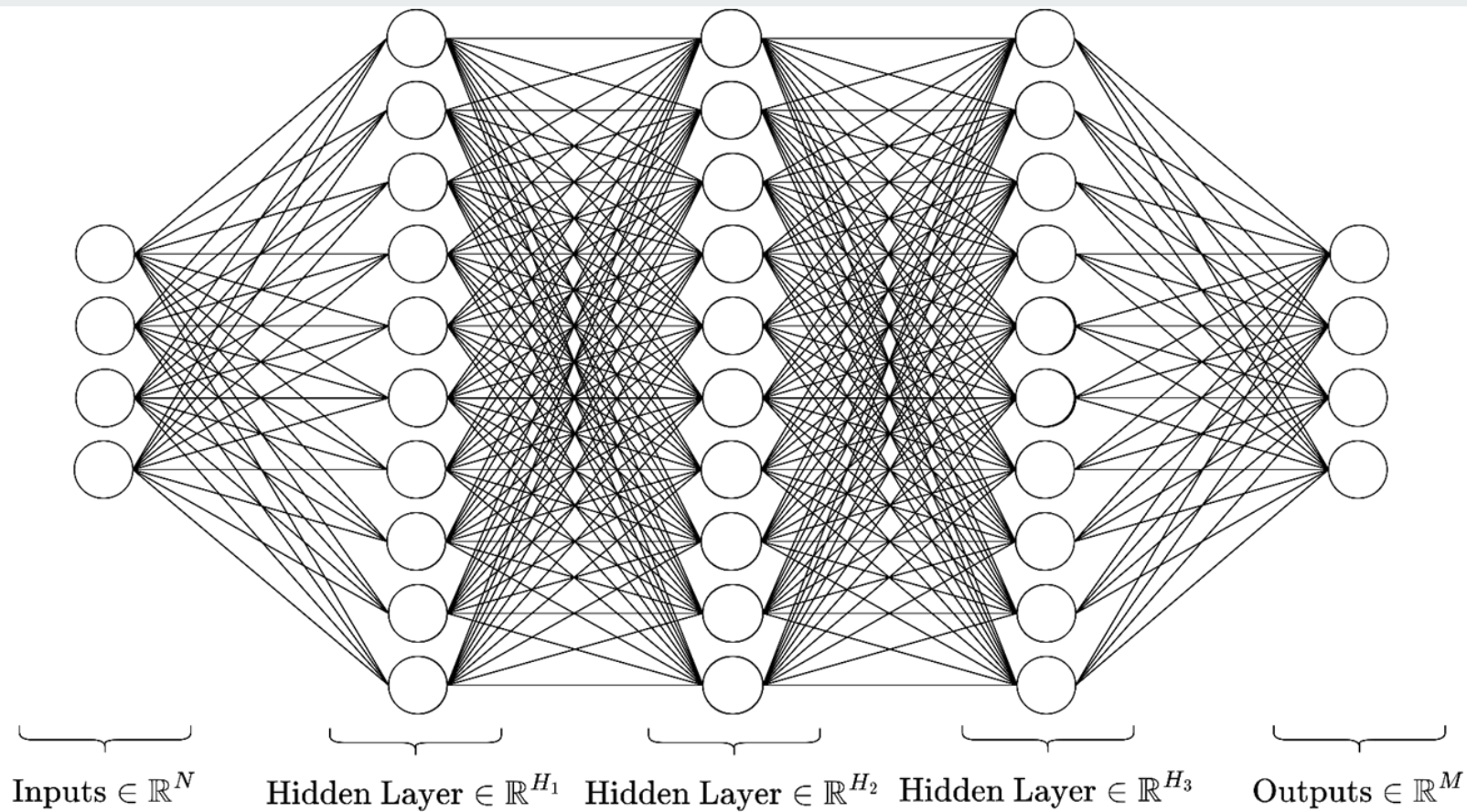


$\underbrace{\hspace{10em}}$
Inputs $\in \mathbb{R}^N$

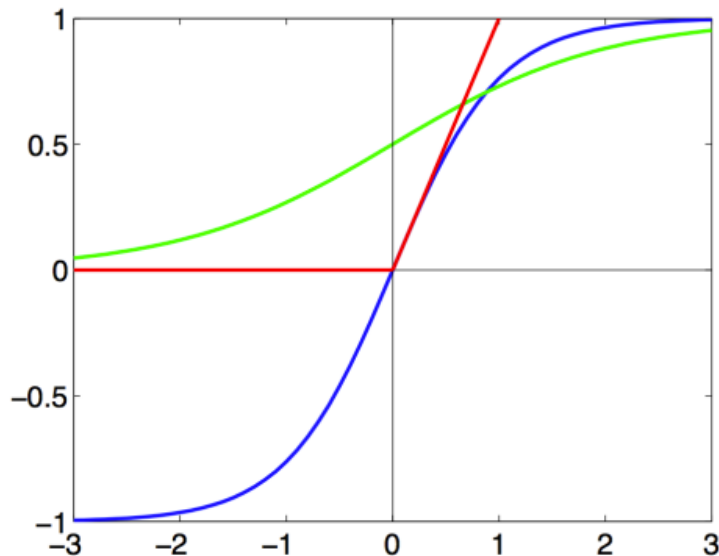








Common nonlinear functions



$$\text{Sigmoid: } \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\text{Hyperbolic tangent: } \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\text{Rectified Linear Unit: } \text{ReLU}(x) = \max(0, x)$$

Sigmoid



$$y = \frac{1}{1 + e^{-x}}$$

Tanh



$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x < n \\ 1, & x \geq n \end{cases}$$

Softplus



$$y = \ln(1 + e^x)$$

ReLU



$$y = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Softsign



$$y = \frac{x}{(1 + |x|)}$$

ELU



$$y = \begin{cases} \alpha(e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid



$$y = \ln\left(\frac{1}{1 + e^{-x}}\right)$$

Swish



$$y = \frac{x}{1 + e^{-x}}$$

Sinc



$$y = \frac{\sin(x)}{x}$$

Leaky ReLU



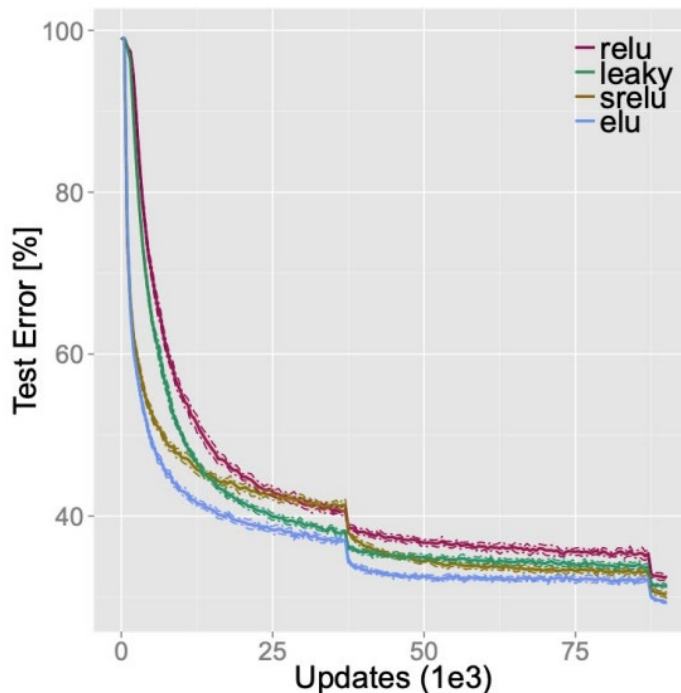
$$y = \max(0.1x, x)$$

Mish



$$y = x(\tanh(\text{softplus}(x)))$$

Image Classification Benchmark (CIFAR-10)



Clevert et al. (2016): Training loss converges fastest with ELU; ELU yields lower test error than ReLU on CIFAR-10



Universal Approximation Theorem

Theorem (schematic). Let $\mathcal{F}$ be a certain class of functions $f : \mathbb{R}^K \rightarrow \mathbb{R}^M$. Then for any $f \in \mathcal{F}$ and any $\varepsilon > 0$ there exists an multilayer perceptron $\mathcal{N}$ with one hidden layer such that $\|f - \mathcal{N}\| < \varepsilon$.

$\Rightarrow$ We can approximate *any* function we want with a *one*-layer MLP!
More effective with more layers than just one (“deeper” networks)

Gradient Descent

General ML Recipe

1. Given training data:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of these:

– Decision function

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

– Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

3. Define goal:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

4. Train with SGD:

(take small steps
opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

Loss functions

Depends on the task

Mean Squared Error (MSE)

Used for regression

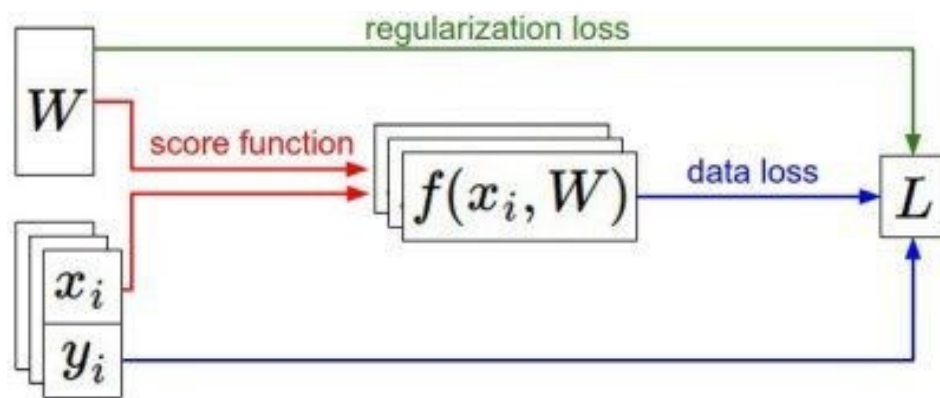
Cross Entropy

Used for classification

Many others

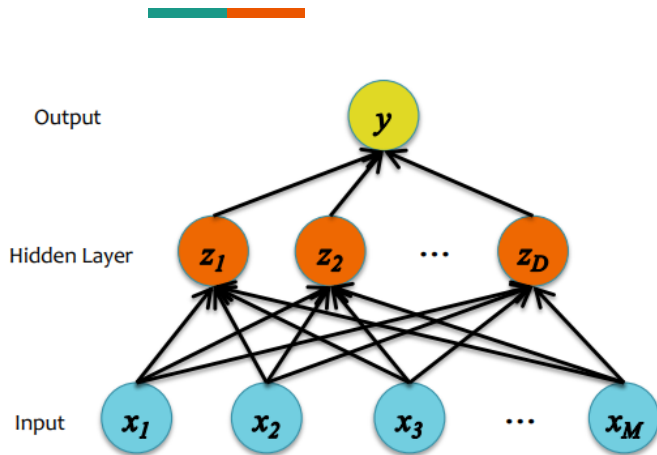
Note: Must be differentiable for gradient descent based optimization

Regularization loss: constrain weights to be sparse or small

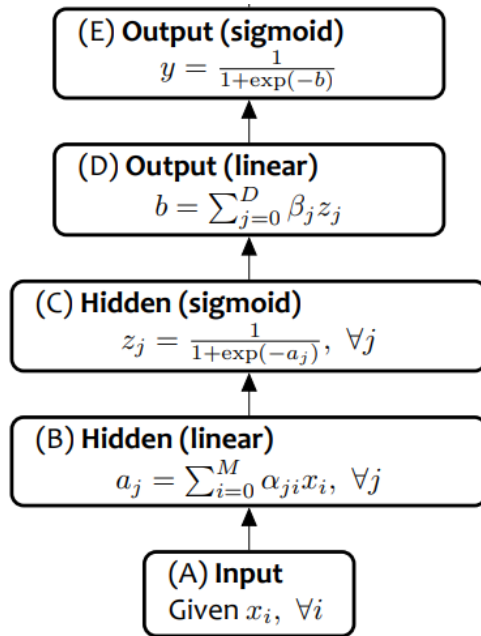


$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$\mathcal{L}_{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

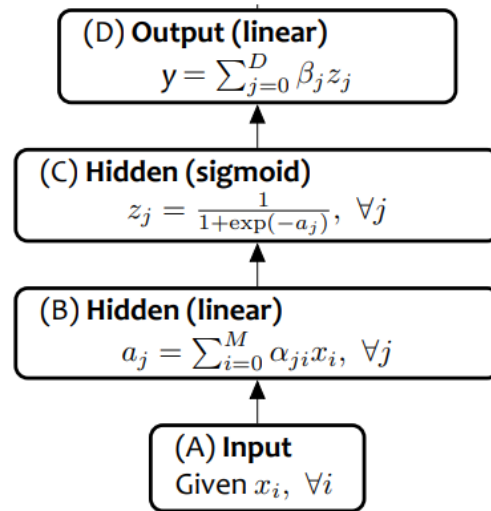


Binary Cross Entropy loss



NN for Binary Classification

MSE loss

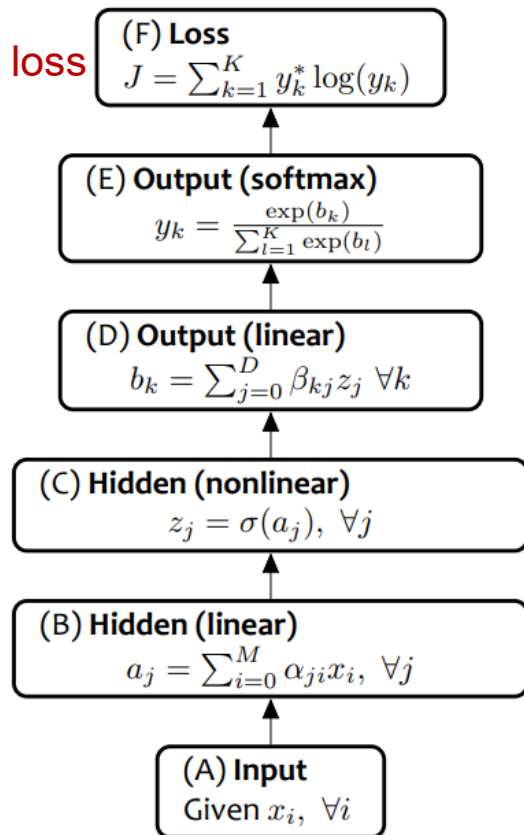
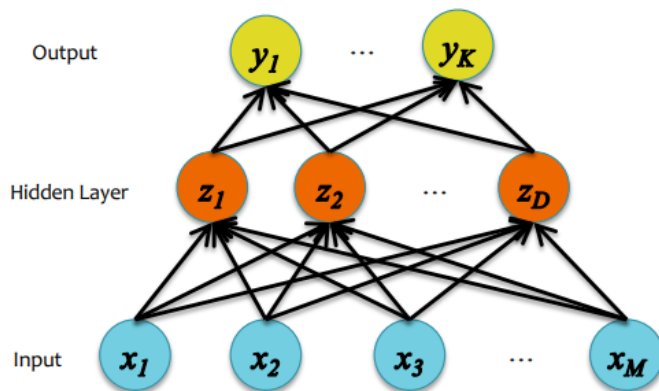


NN for Regression

Softmax:

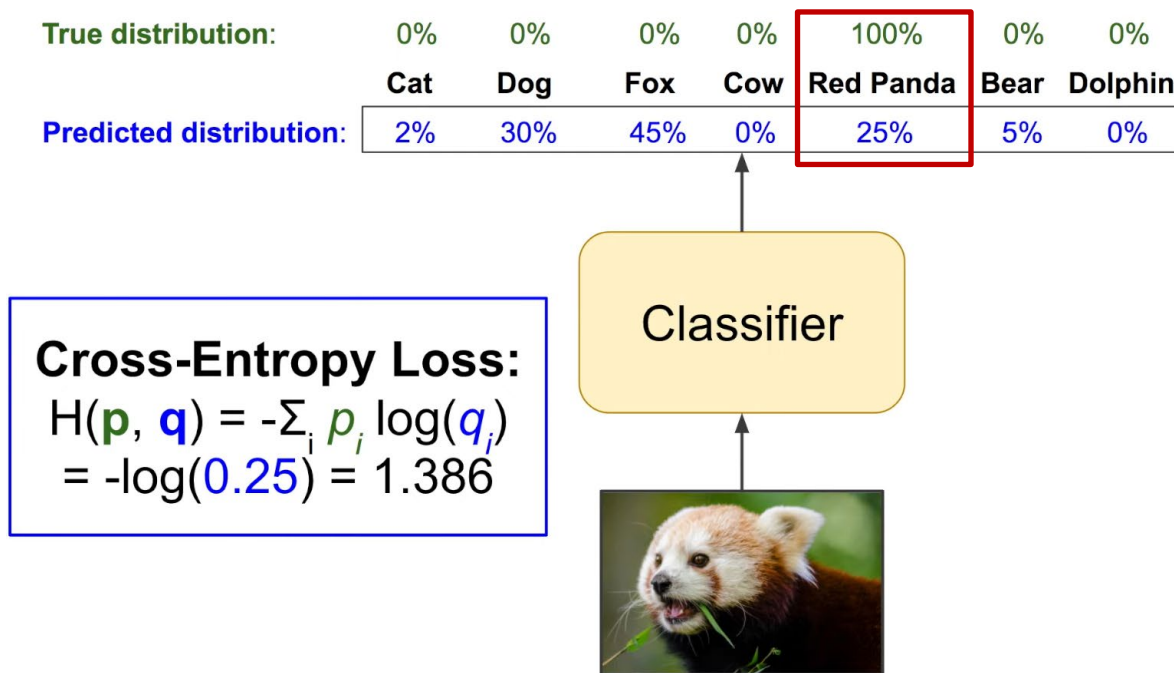
Cross Entropy loss

$$y_k = \frac{\exp(b_k)}{\sum_{l=1}^K \exp(b_l)}$$



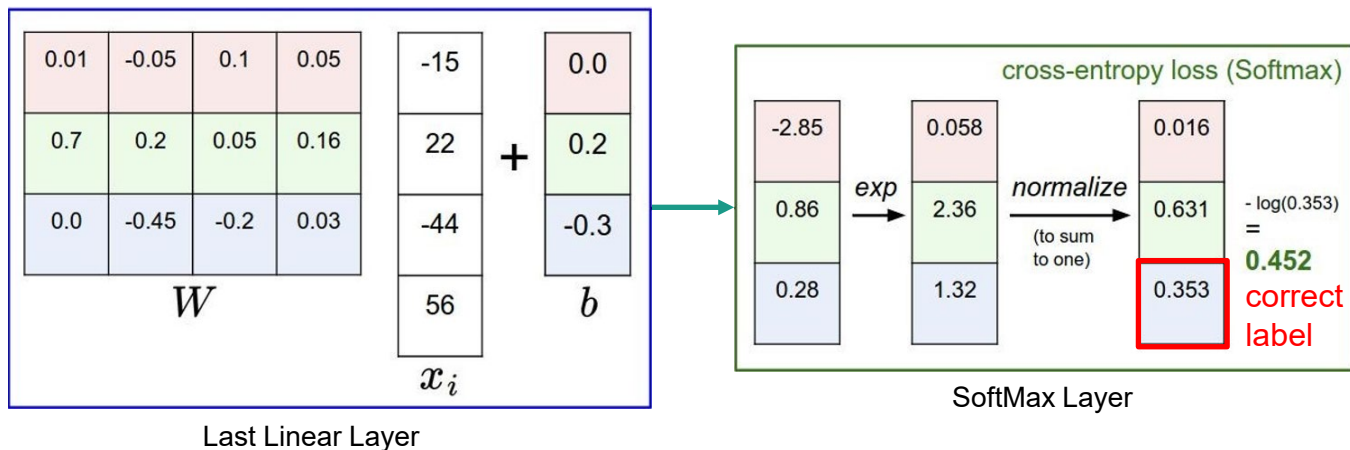
Cross Entropy Loss

For a given training sample, CE loss measures how much the **predicted distribution** diverges from the **true distribution**, which is a one-hot vector: the entry corresponding to true label being 1 and all others 0.



Cross Entropy Loss Example

- Consider a NN for 3-class classification. The last linear layer computes the vector of logits $h_{\theta}(x) = Wx_i + b = [-2.85 \quad .86 \quad .28]^T$ (x is the input image, x_i is the intermediate input to the last layer)
- The SoftMax layer computes the vector of predicted probabilities $[.016 \quad .631 \quad .353]^T$ for labels $[1 \quad 2 \quad 3]^T$
 - Logits: $[e^{-2.85}, e^{.86}, e^{.28}] = [.058, 2.36, 1.32]$
 - Normalize by $e^{-2.85} + e^{.86} + e^{.28} = 3.738$ to get SoftMax scores $[\frac{.058}{3.738}, \frac{2.36}{3.738}, \frac{1.32}{3.738}] = [.016, .631, .353]$
- CE loss is thus $-\log .353$, assuming correct label $y_i = 3$





Implementing learning: Gradient Descent

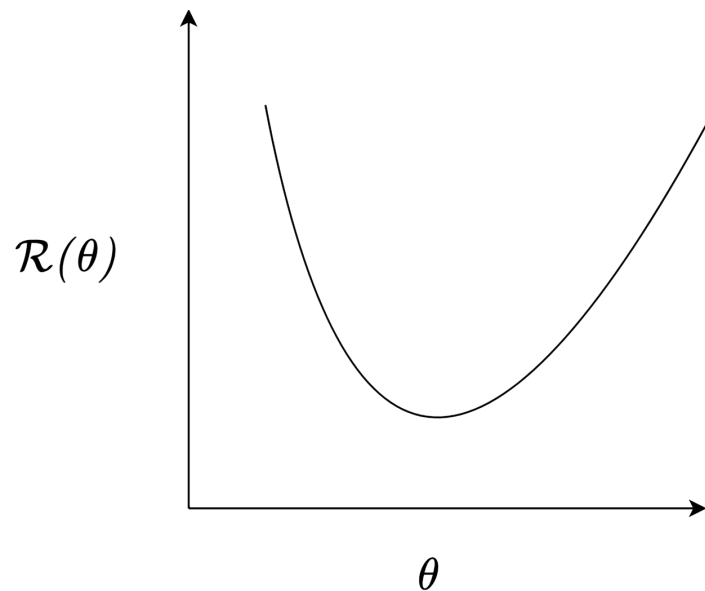
Given:

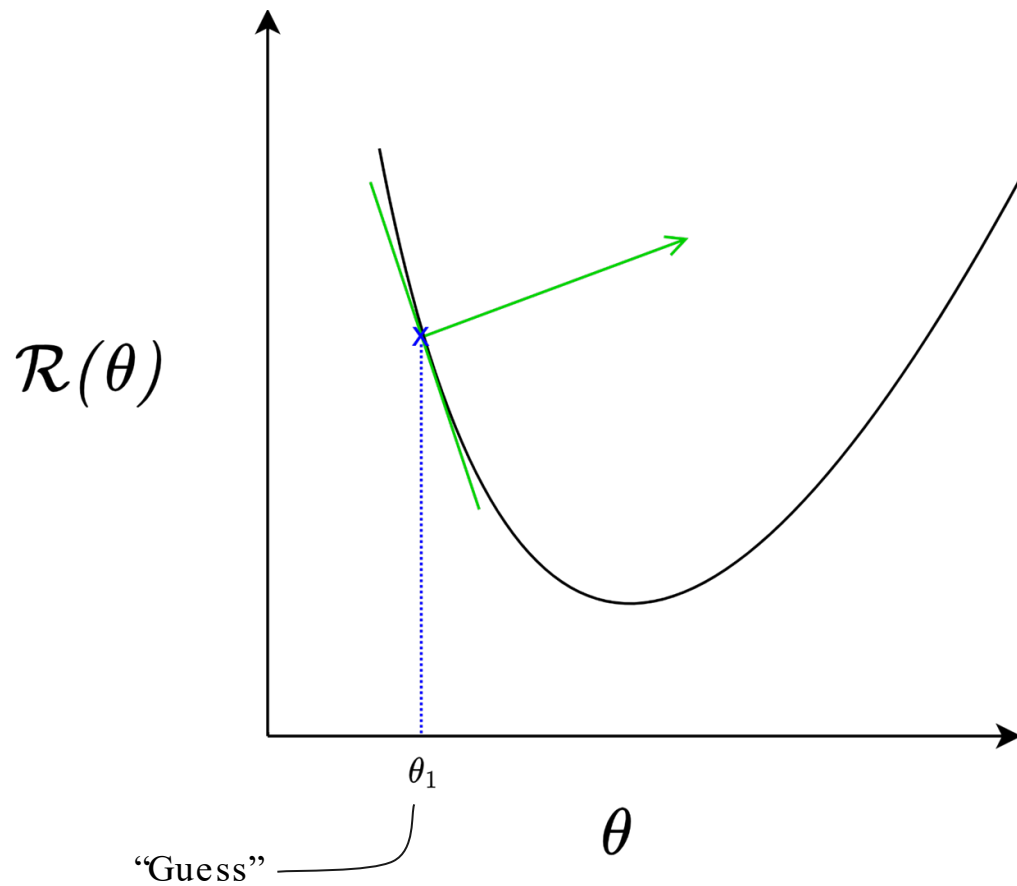
- Family of parameters Θ (e.g. possible weights of a NN)
- Differentiable risk function (loss function computed over a set of samples) $\mathcal{R}(\theta)$

Goal:

$$\theta_{opt} = \operatorname{argmin}_{\theta \in \Theta} \mathcal{R}(\theta)$$

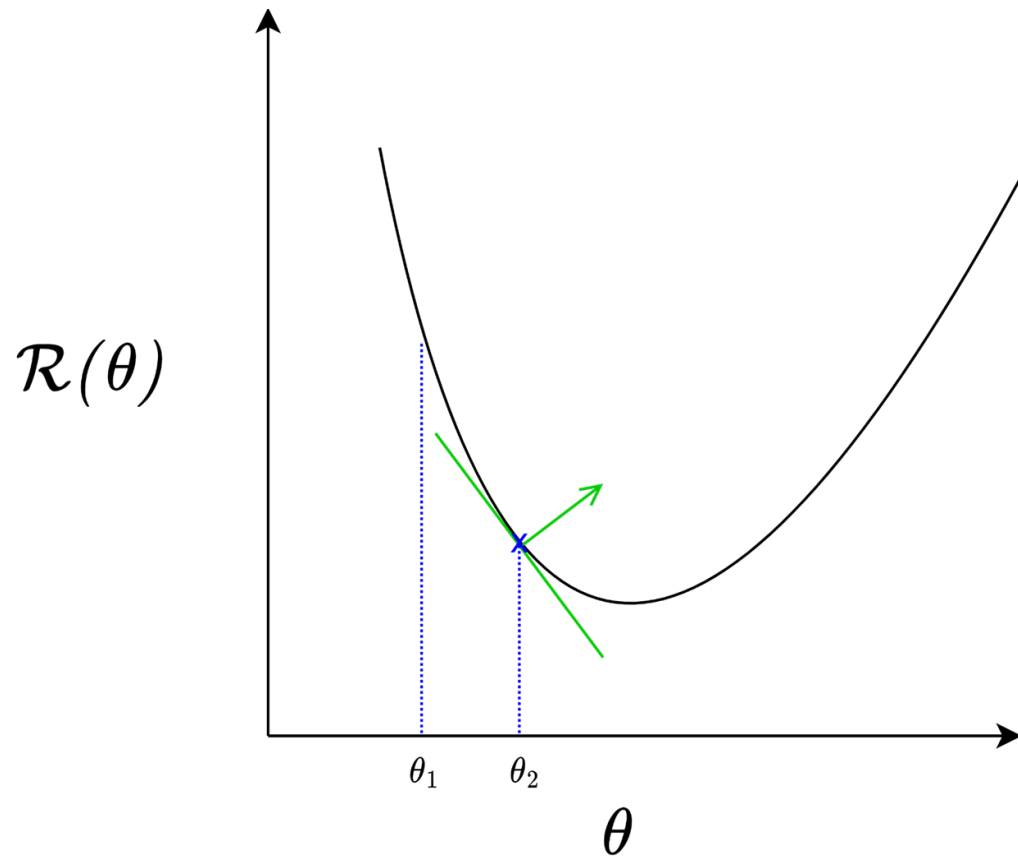
Backprop: Gradient descent



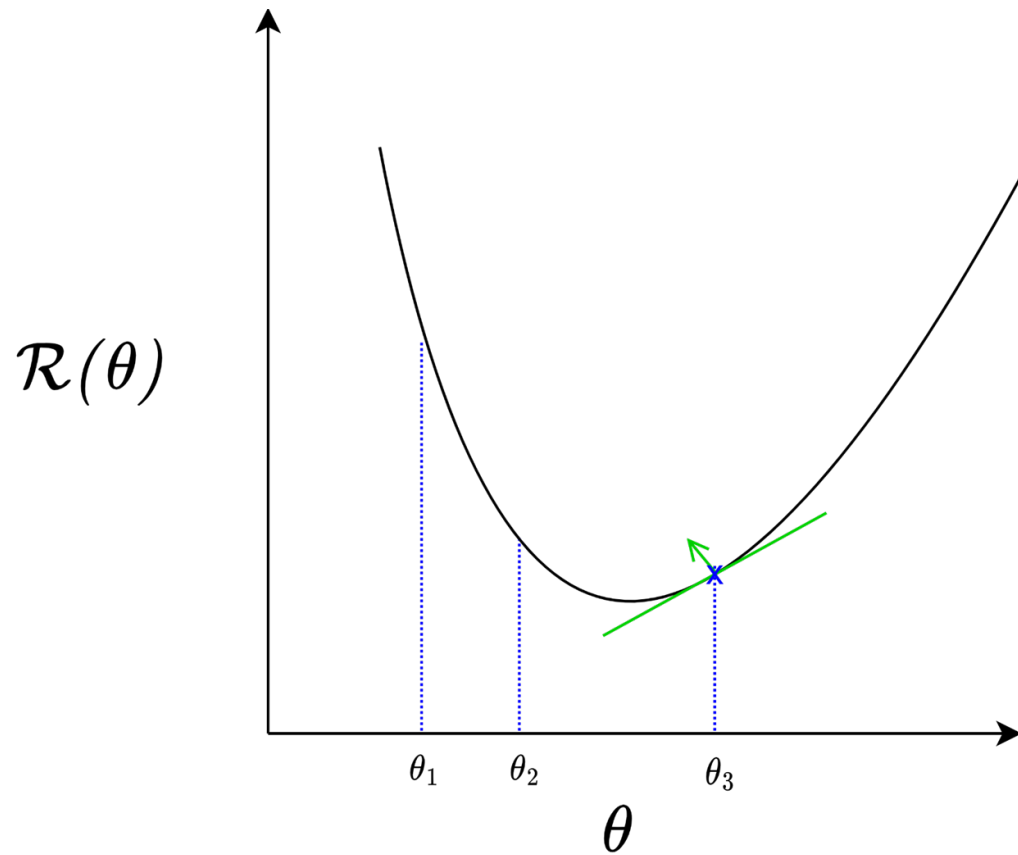


"Learning rate"

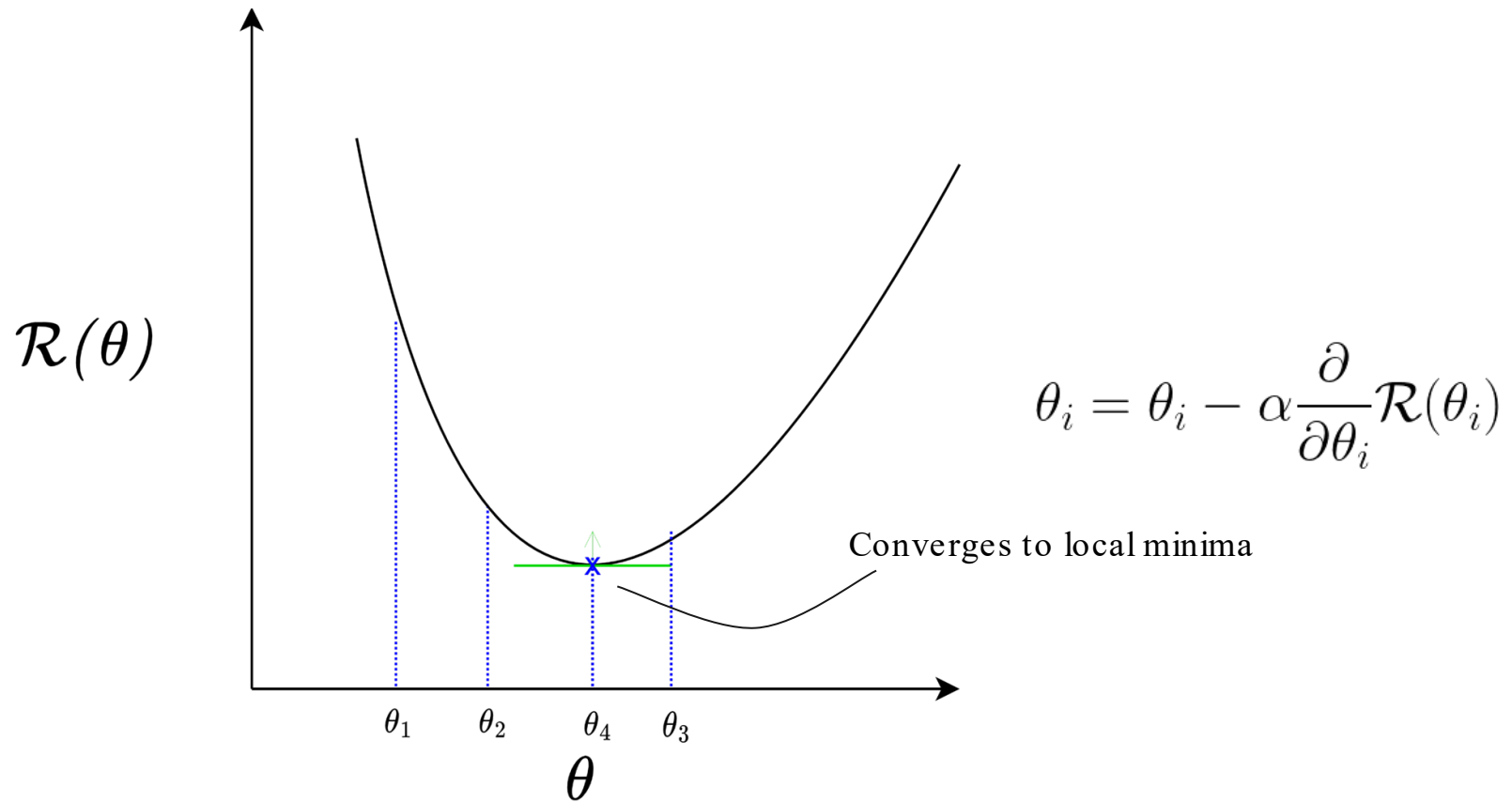
$$\theta_i = \theta_i - \alpha \frac{\partial}{\partial \theta_i} \mathcal{R}(\theta_i)$$



$$\theta_i = \theta_i - \alpha \frac{\partial}{\partial \theta_i} \mathcal{R}(\theta_i)$$

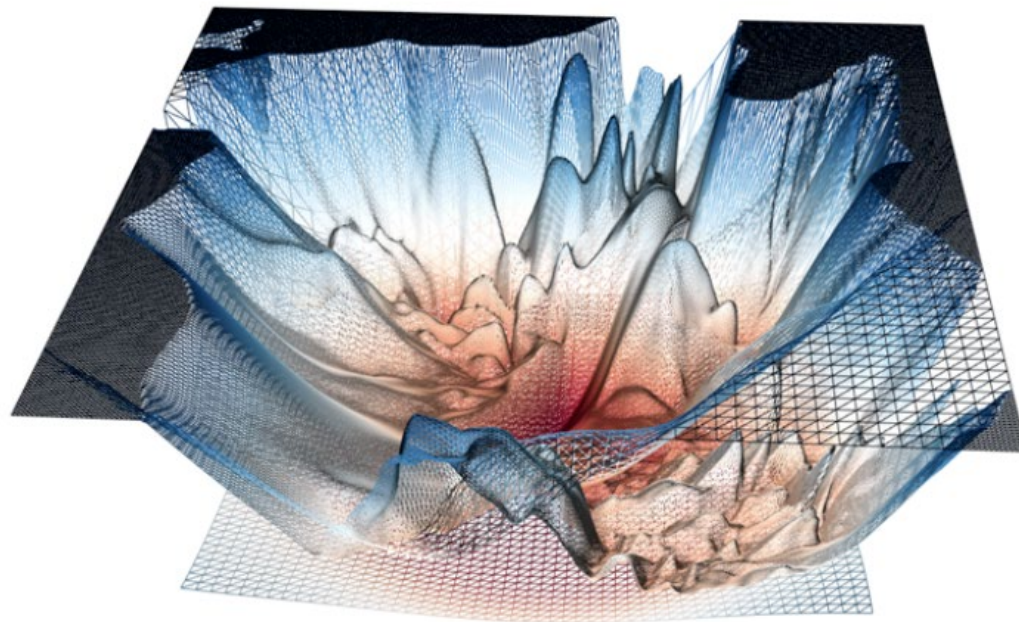


$$\theta_i = \theta_i - \alpha \frac{\partial}{\partial \theta_i} \mathcal{R}(\theta_i)$$



A “real” loss landscape:

- Many (many many) local minima
- Saddle points



<http://www.telesens.co/2019/01/16/neural-network-loss-visualization/>

$$\frac{d\mathcal{L}}{dw_l} = \frac{d\mathcal{L}}{dh_L} \cdot \frac{dh_L}{dh_{L-1}} \cdot \dots \cdot \frac{dh_l}{dw_l} \Rightarrow \frac{d\mathcal{L}}{dw_l} = \frac{d\mathcal{L}}{dh_l} \cdot \frac{dh_l}{dw_l}$$

Gradient of loss w.r.t. the module output Gradient of a module w.r.t. its parameters

Backprop: Efficient NN GD

Apologies for different notation

- Goal: change optimization from $\mathcal{O}(|\theta|^2)$ to $\mathcal{O}(|\theta|)$
- Recall an MLP:

$$\mathcal{N}(x) = a^\ell \circ h^\ell \circ a^{\ell-1} \circ h^{\ell-1} \circ \dots \circ a^1 \circ h^1 x$$

$$\mathcal{L}(\mathcal{N}(x), y) \longrightarrow \frac{\partial \mathcal{L}(\mathcal{N}(x), y)}{\partial w_i} \longrightarrow w_i \leftarrow w_i - \eta \frac{\partial \mathcal{L}(\mathcal{N}(x), y)}{\partial w_i}$$

- Use the chain rule to compute the derivatives from output to input
“Backpropagation of errors”

Gradient Descent

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

PyTorch builds a **computational graph** during the forward pass; then `loss.backward()` uses chain rule to compute analytic gradient exactly.

Example: For the function $y = x^2 + 3x$,

PyTorch computes $\frac{dy}{dx} = 2x + 3$. With $x = 2$,
 $x.grad = \frac{dy}{dx} = 7$.

```
import torch
x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 3*x
y.backward()
print(x.grad)
# Output:tensor(7.)
```

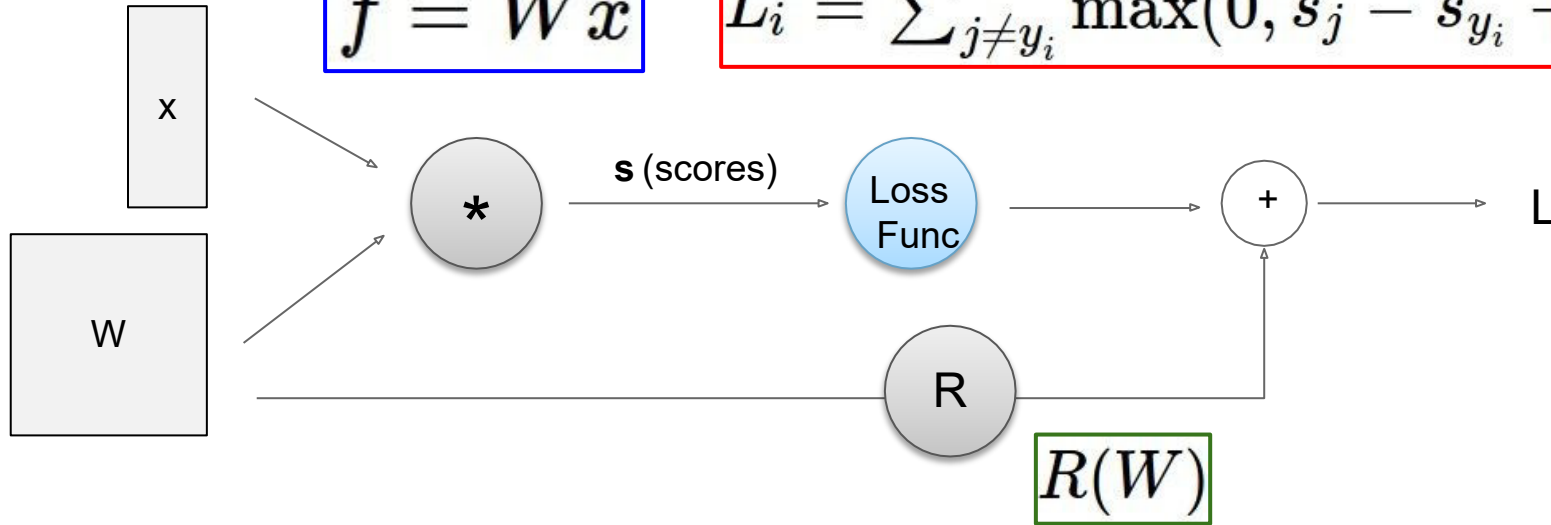
Computational Graph Example

Feature Encoding

$$f = Wx$$

Hinge Loss

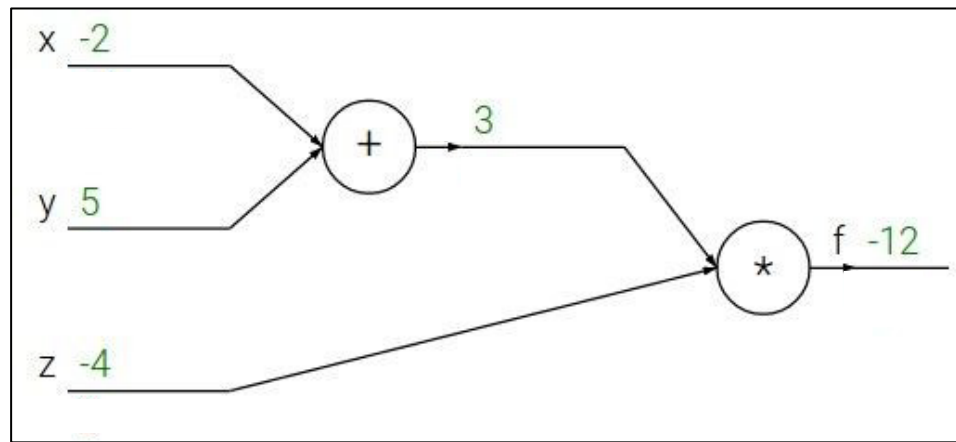
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



Weight Regularization (L2 or L1)

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



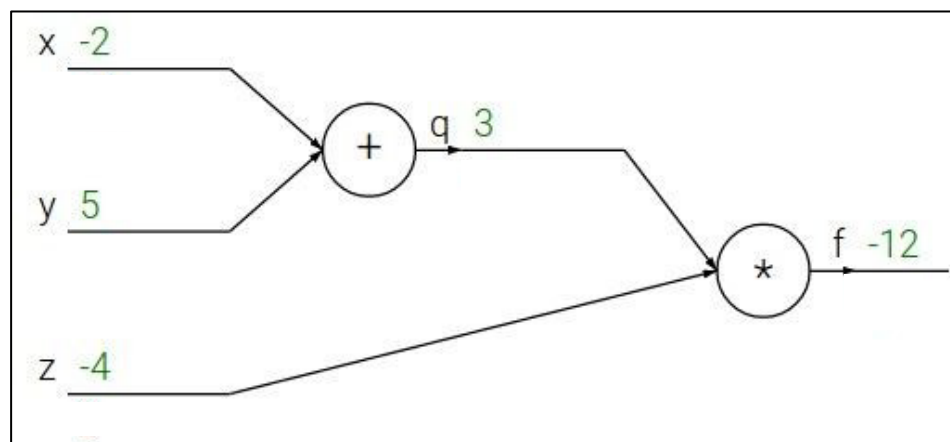
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



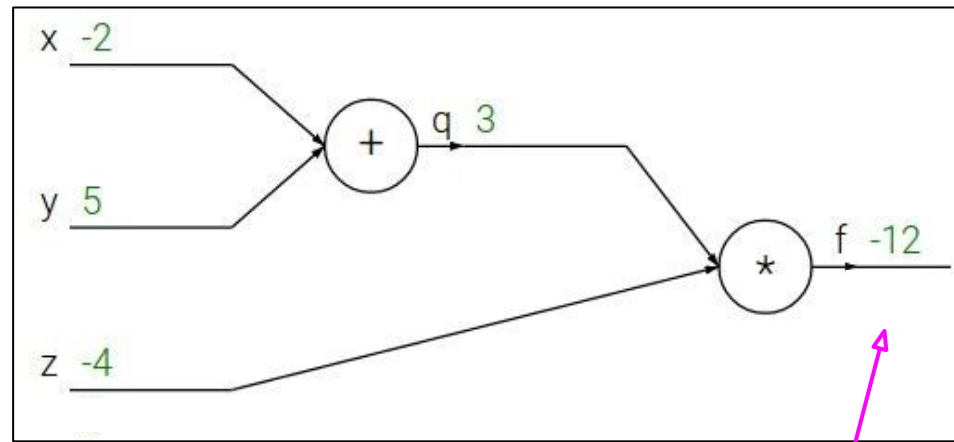
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

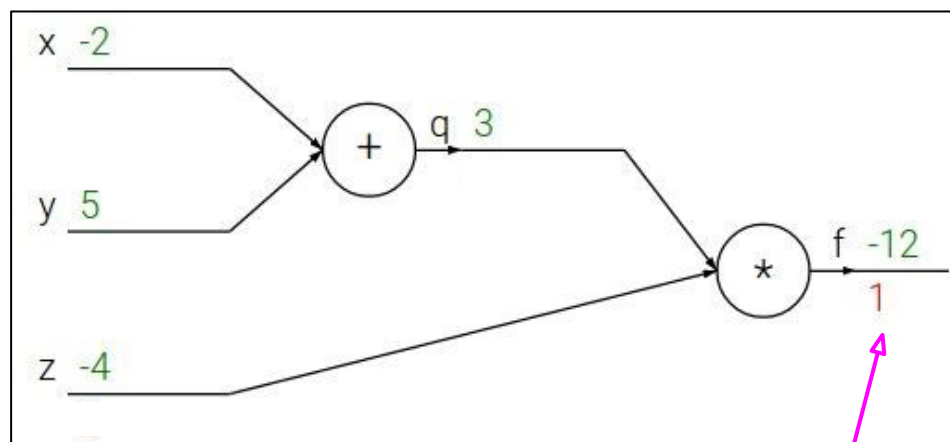
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

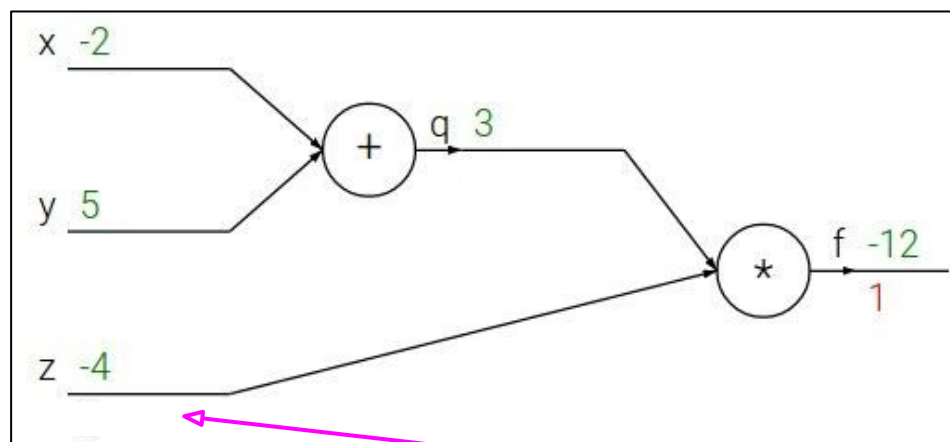
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

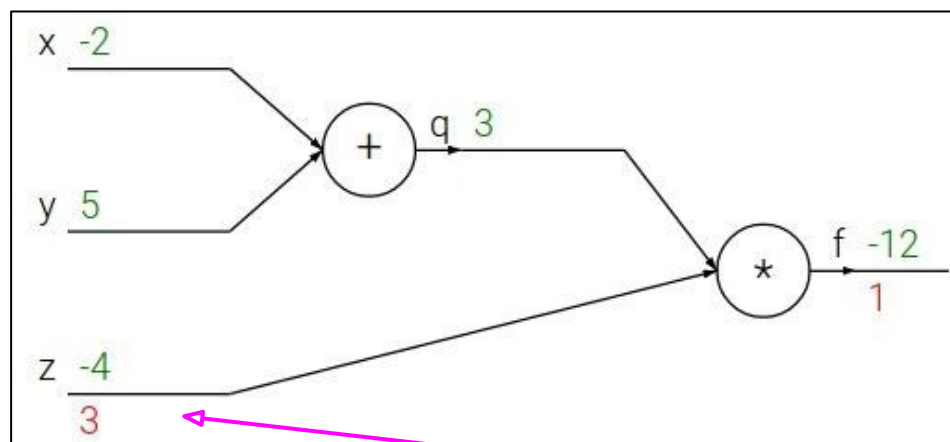
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

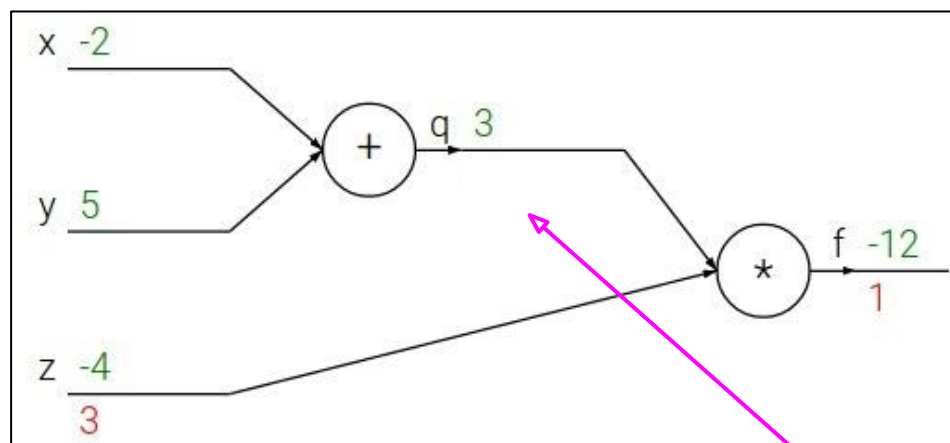
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



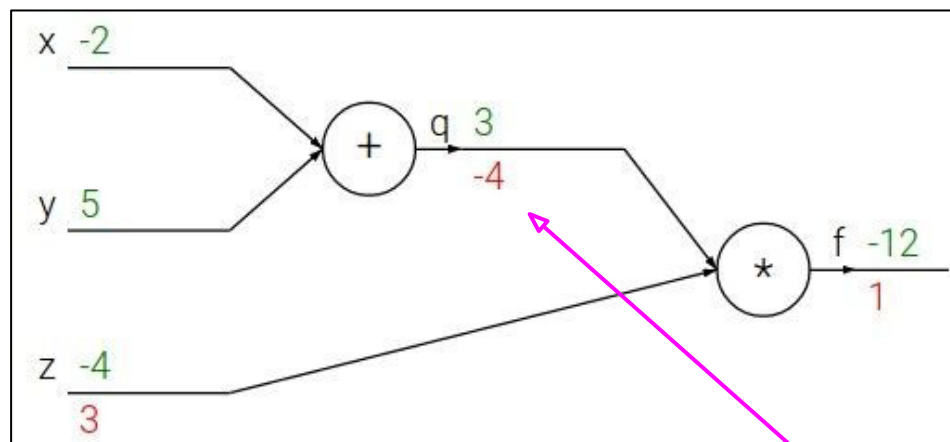
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

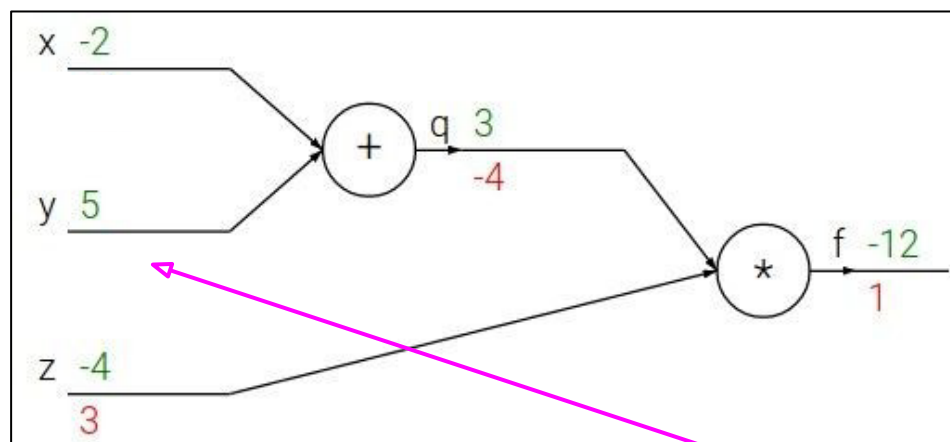
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

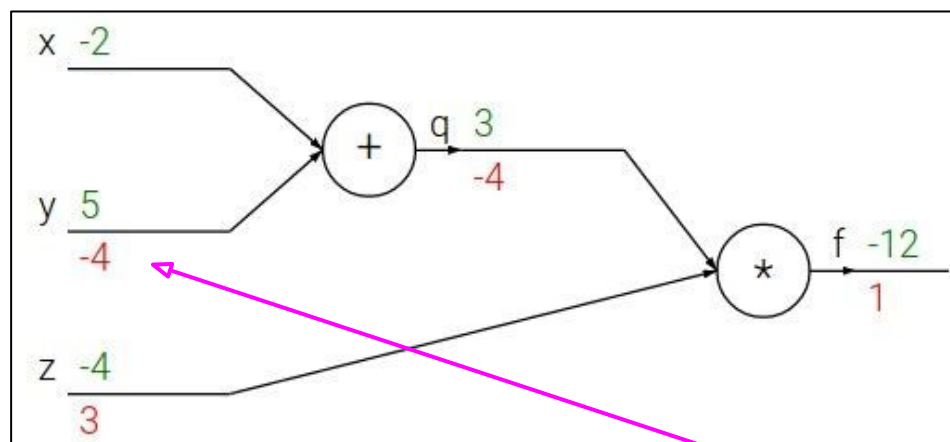
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

-4 1

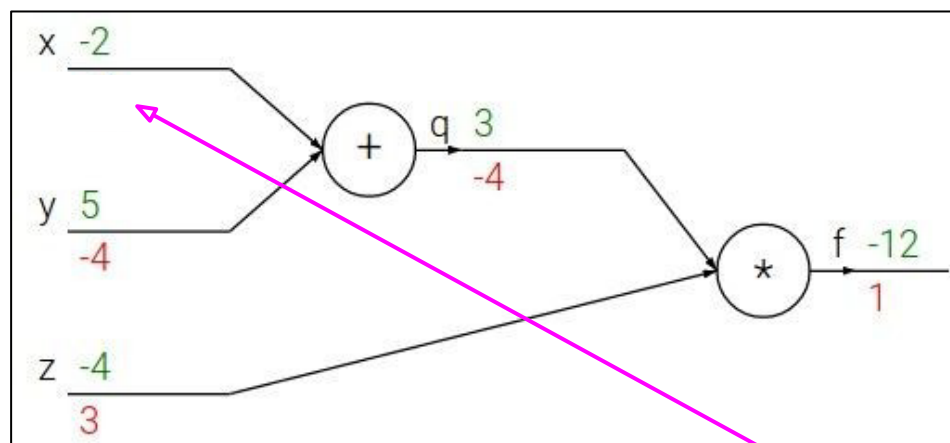
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

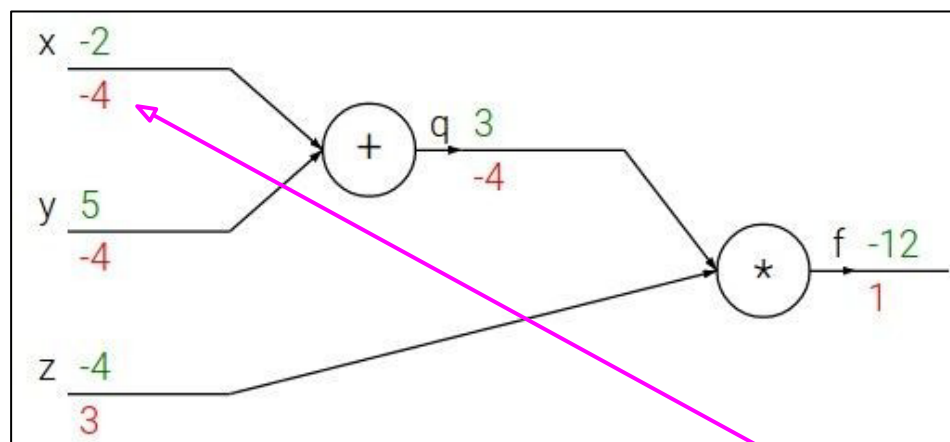
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



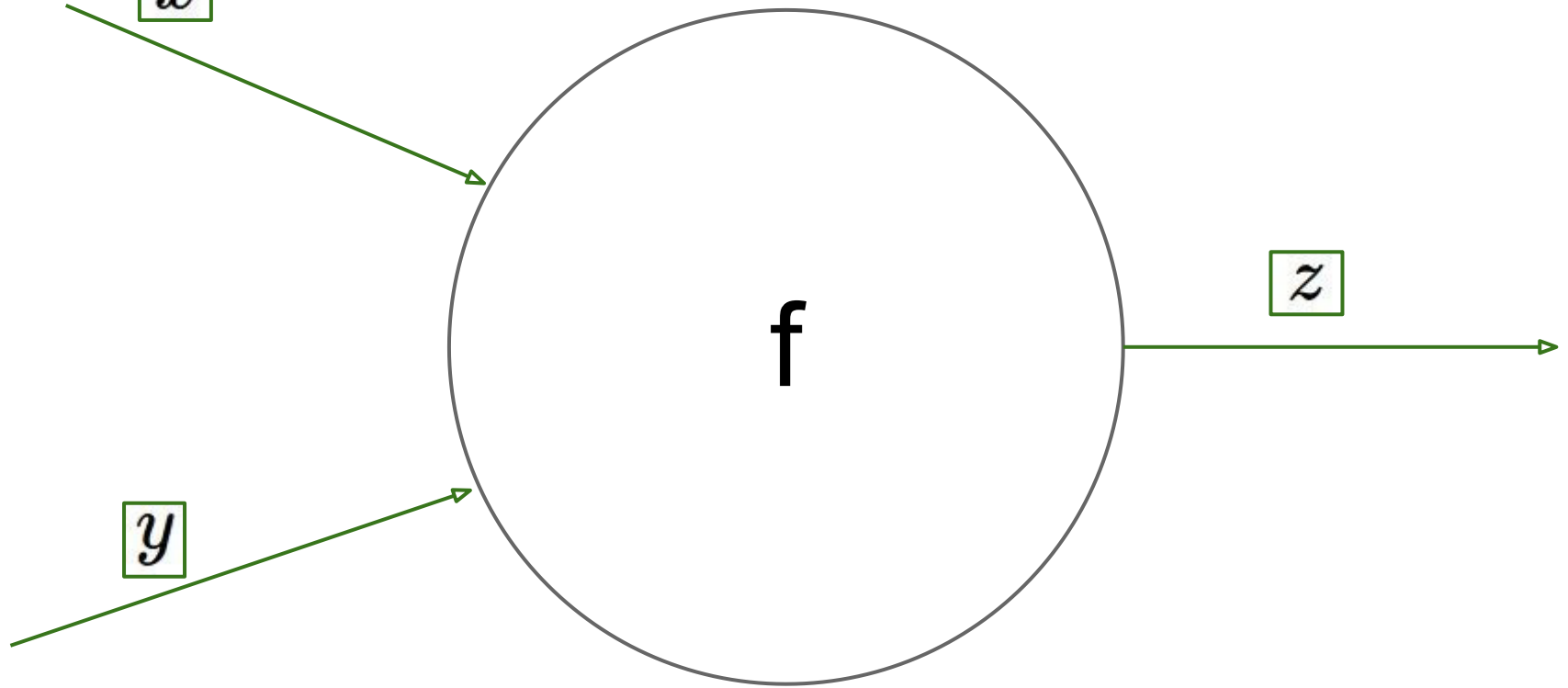
$$\frac{\partial f}{\partial x}$$

Chain rule:

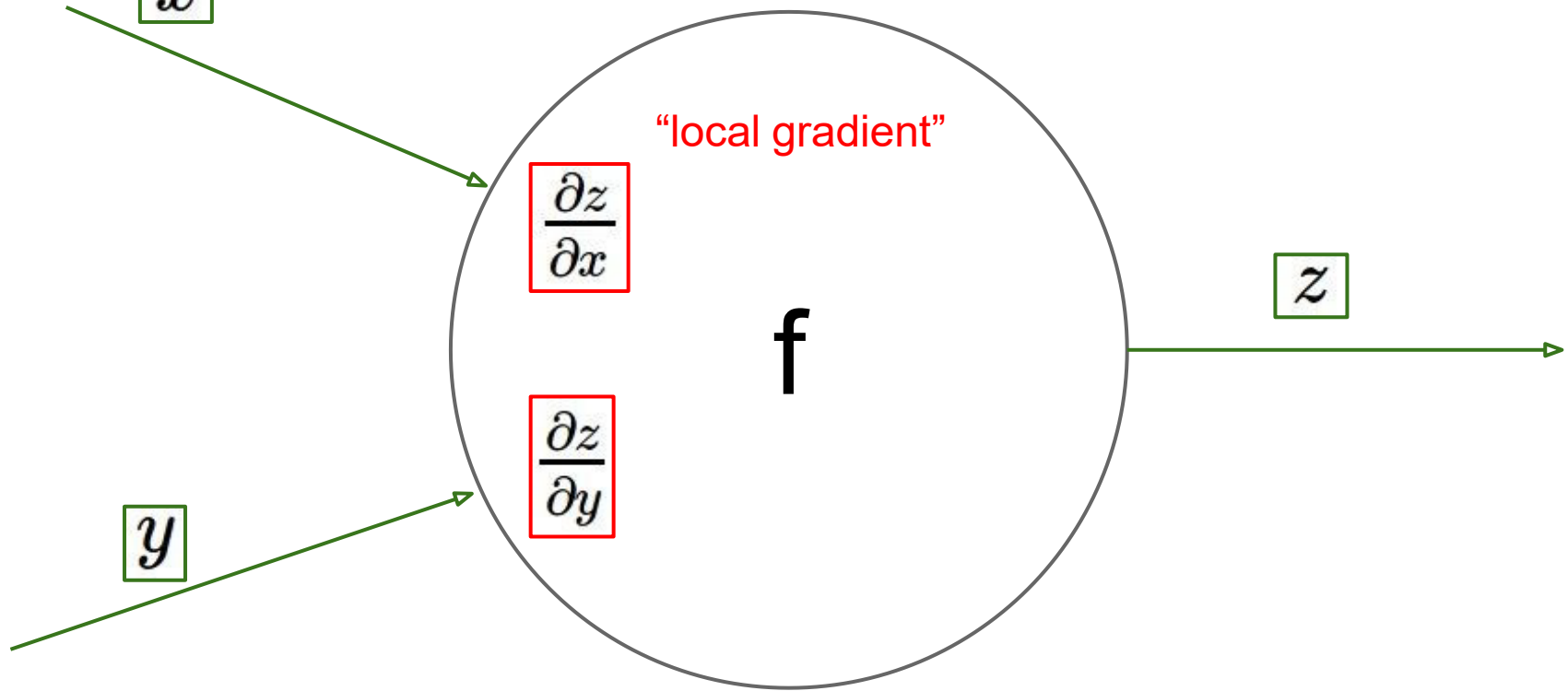
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

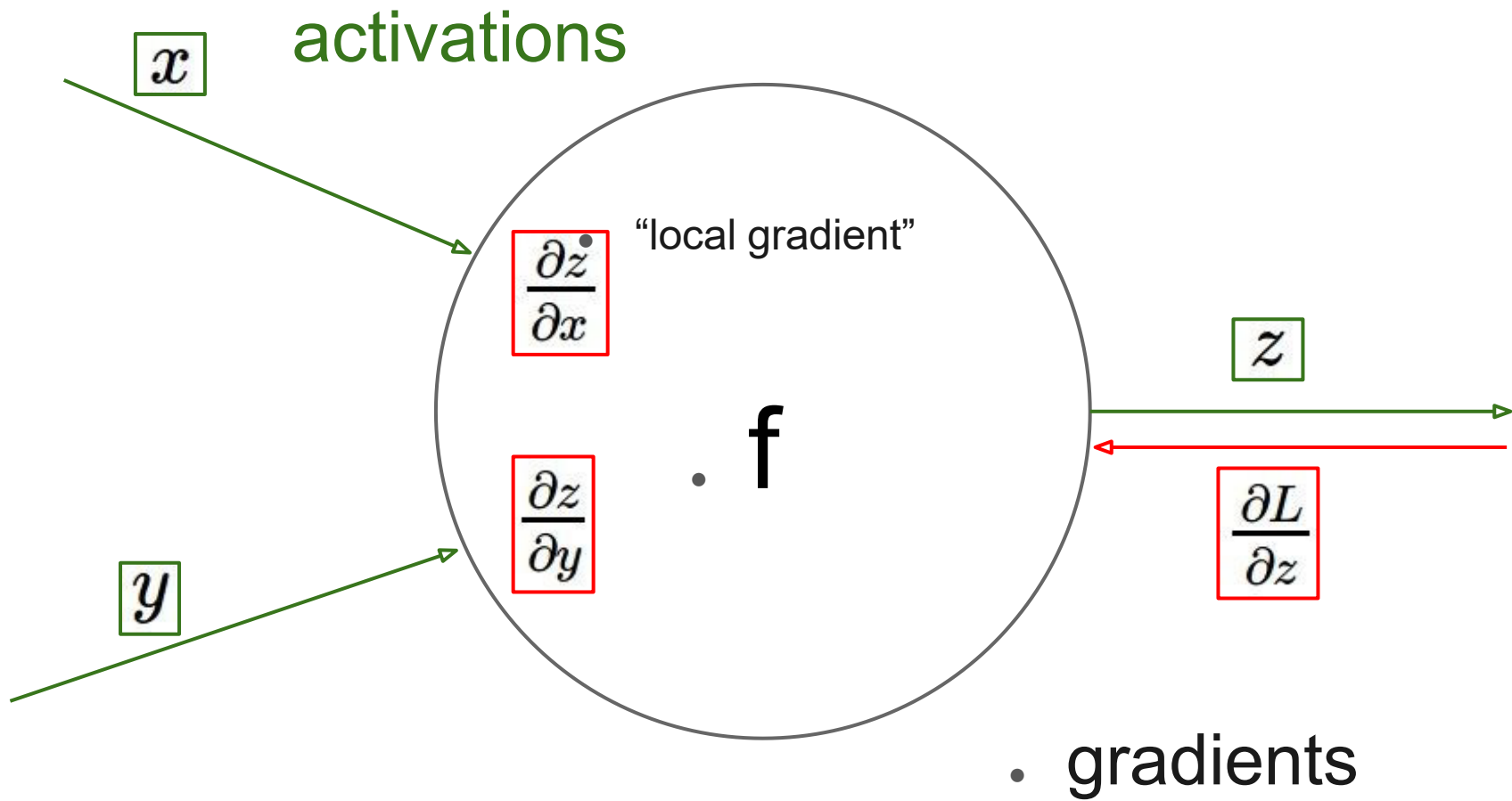
$\uparrow$ $\uparrow$
 -4 1

activations

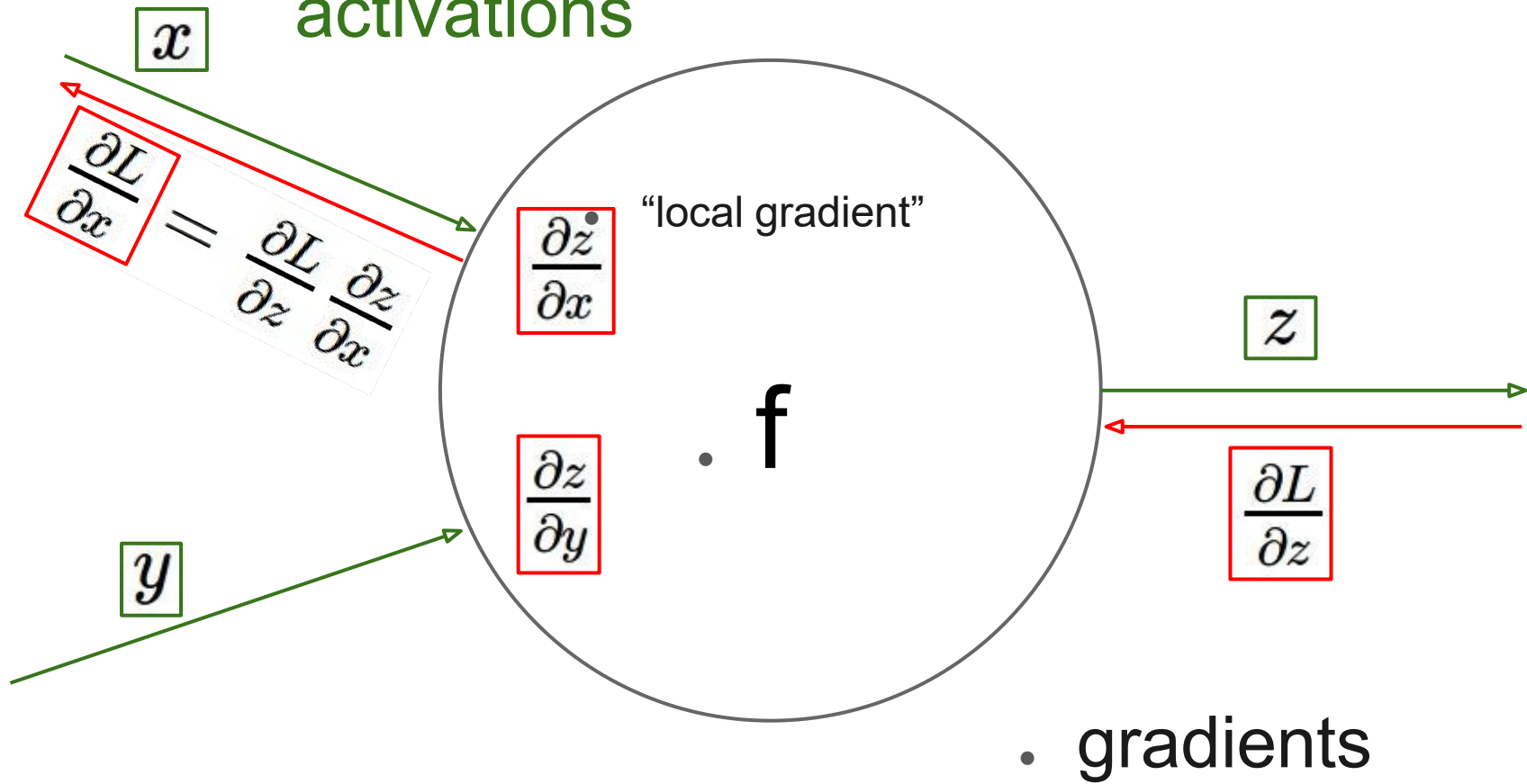


activations

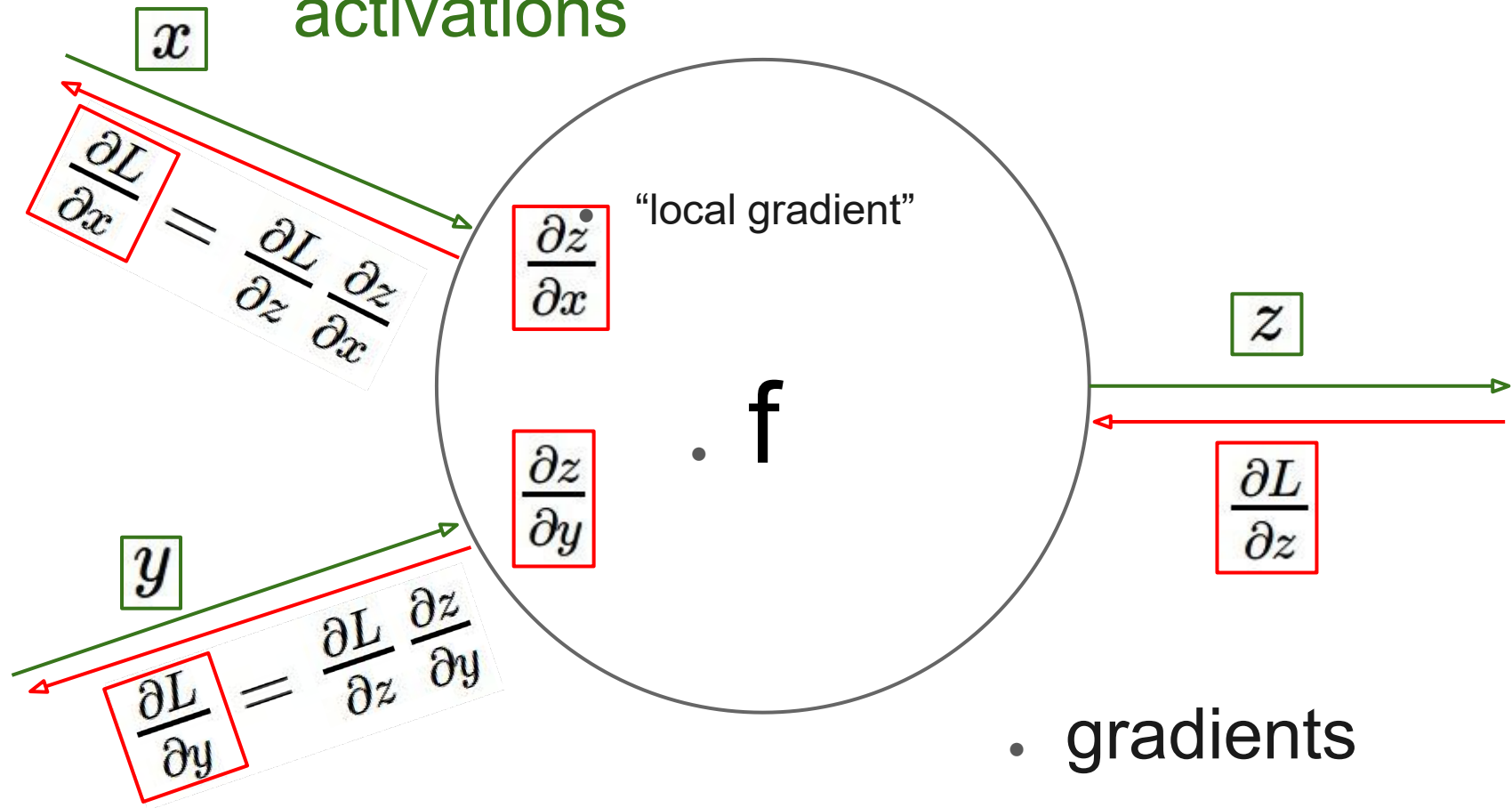




activations



activations



activations

x

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

$$\frac{\partial z}{\partial x}$$

“local gradient”

$$\frac{\partial z}{\partial y}$$

f

z

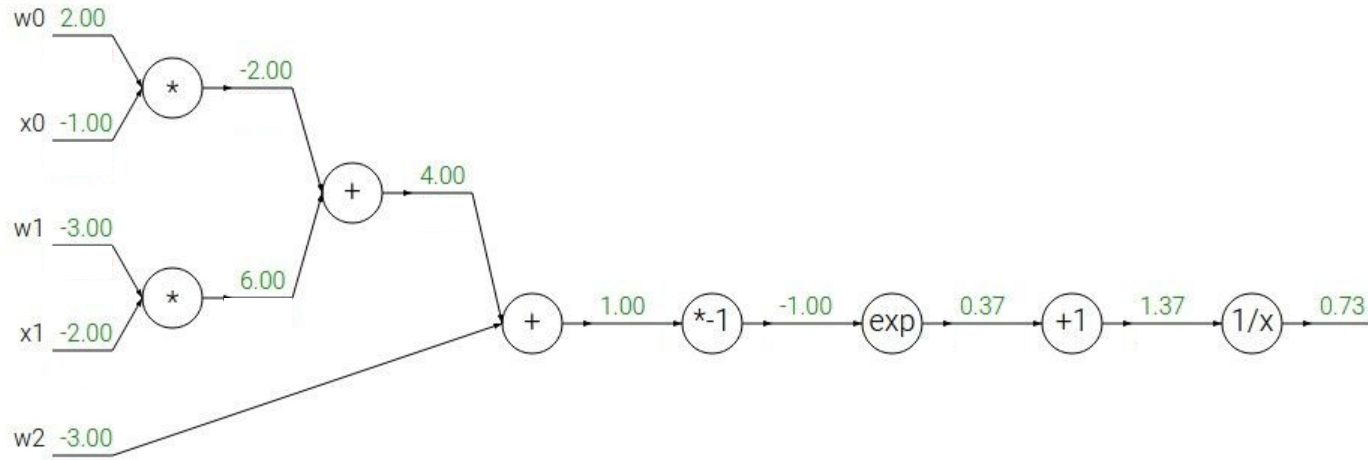
$$\frac{\partial L}{\partial z}$$

y

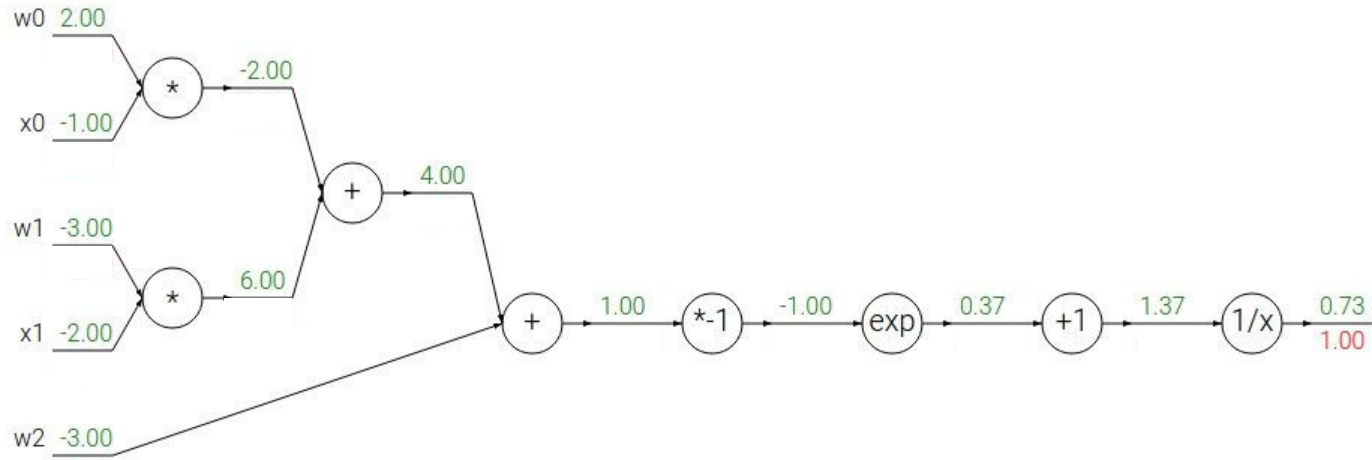
$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial y}$$

gradients

Another example:
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

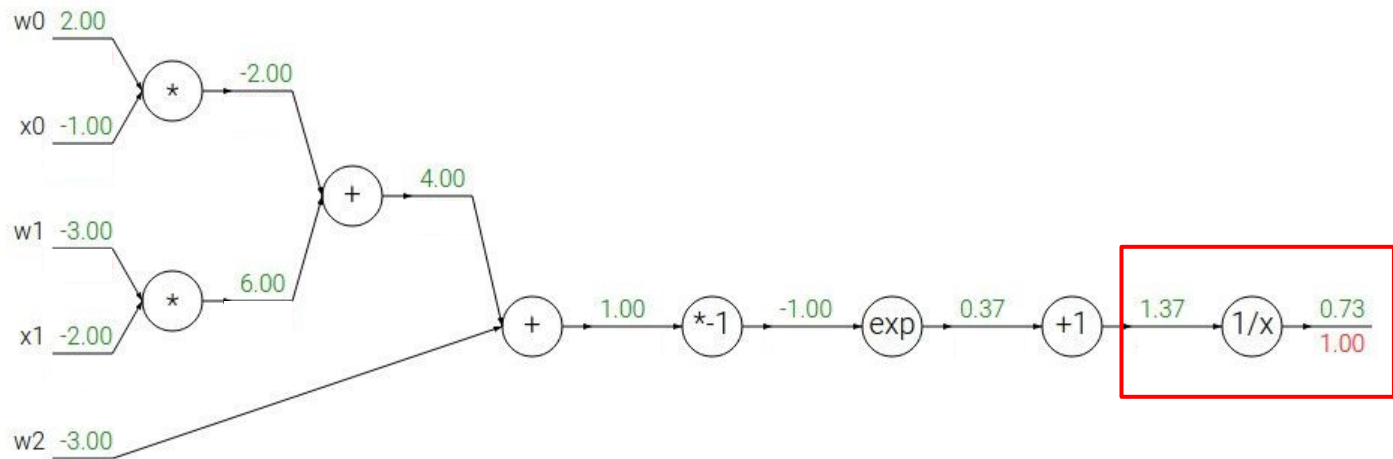


$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$	 	$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$	 	$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x$$

 $\rightarrow$

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

 $\rightarrow$

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

 $\rightarrow$

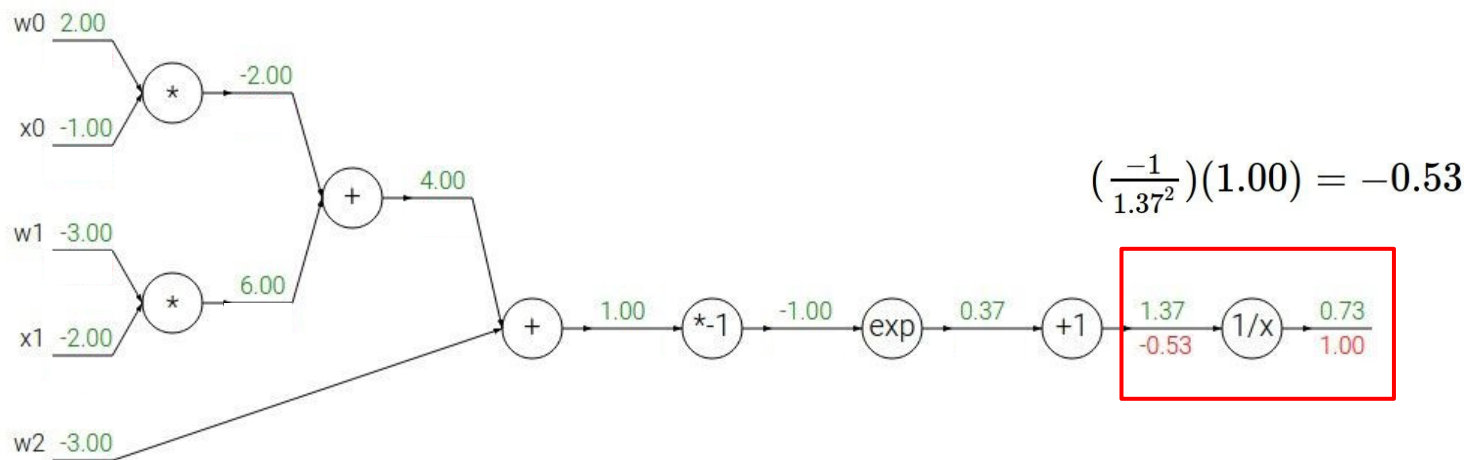
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

 $\rightarrow$

$$\frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

→

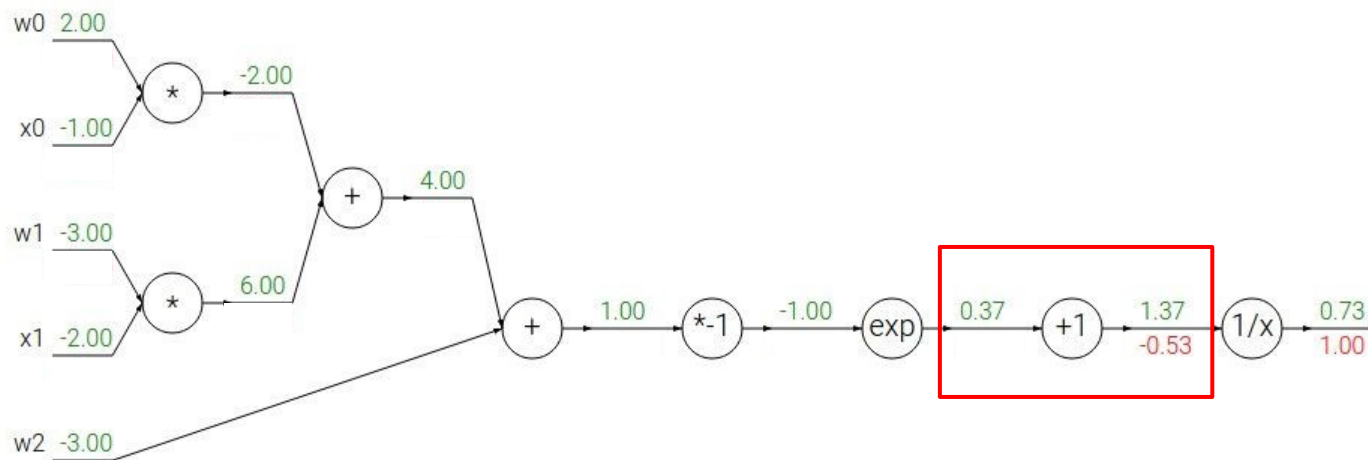
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

→

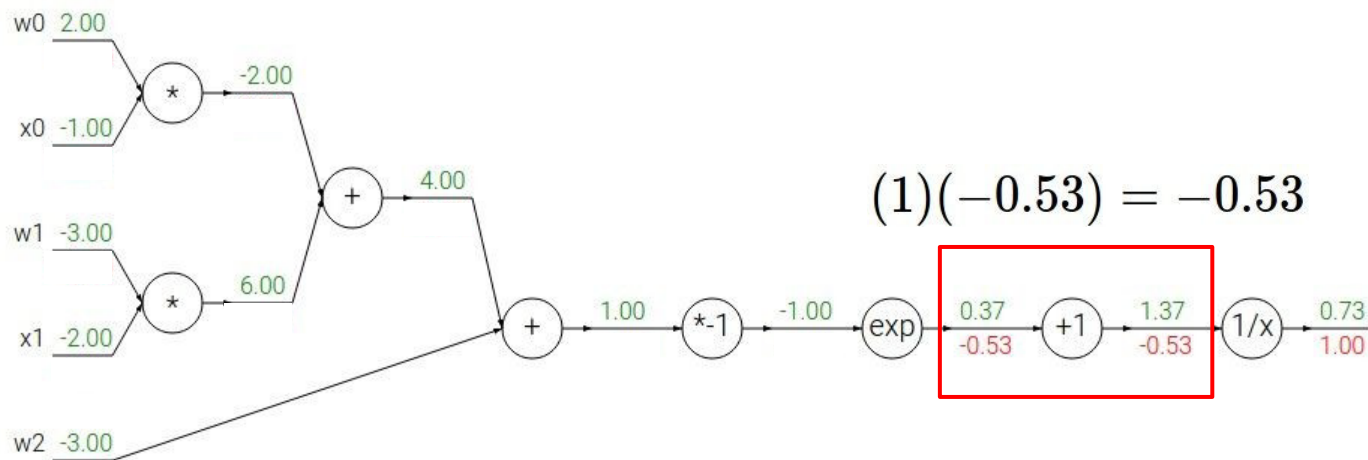
$$\frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$(1)(-0.53) = -0.53$$

$$f(x) = e^x$$

 $\rightarrow$

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

 $\rightarrow$

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

 $\rightarrow$

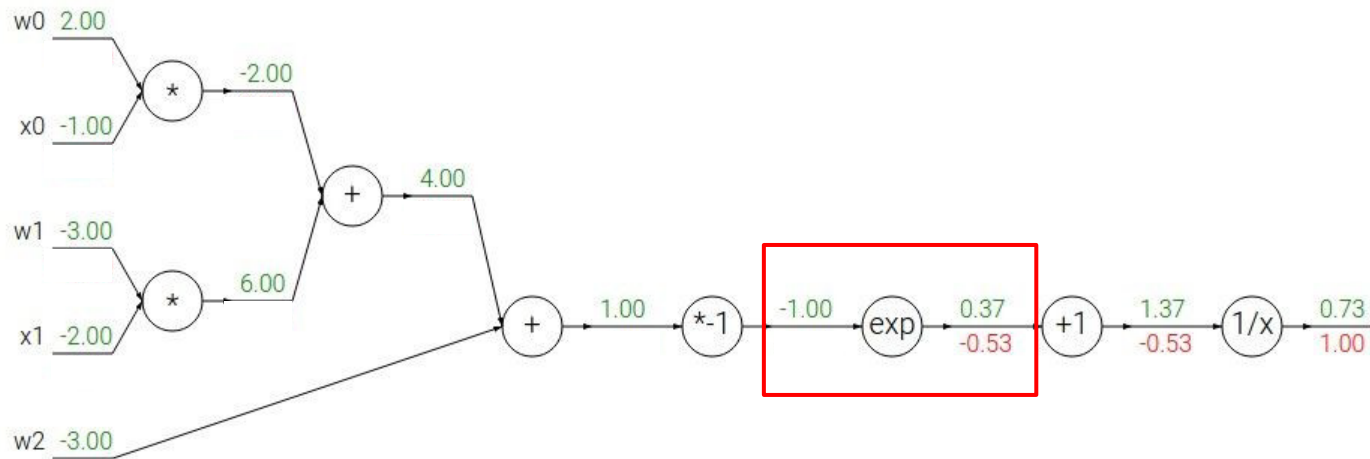
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

 $\rightarrow$

$$\frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



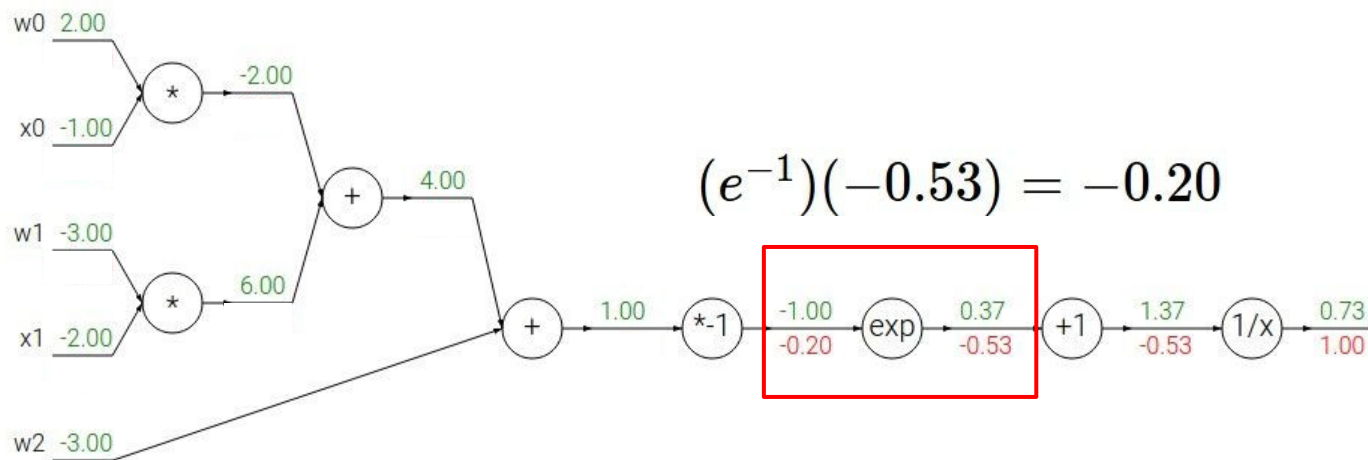
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$(e^{-1})(-0.53) = -0.20$$

$$f(x) = e^x$$

 $\rightarrow$

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

 $\rightarrow$

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

 $\rightarrow$

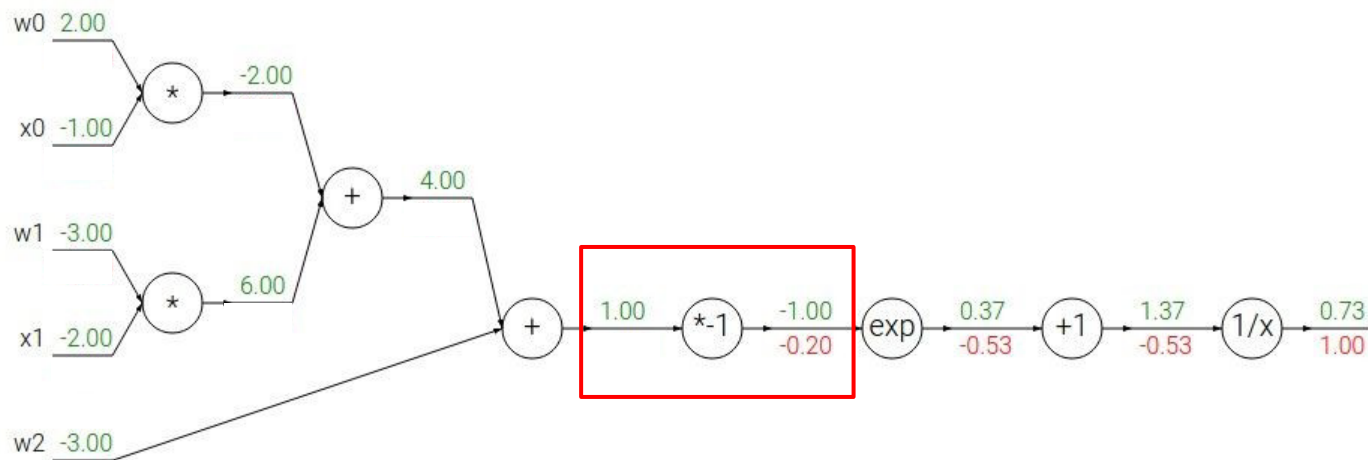
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

 $\rightarrow$

$$\frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



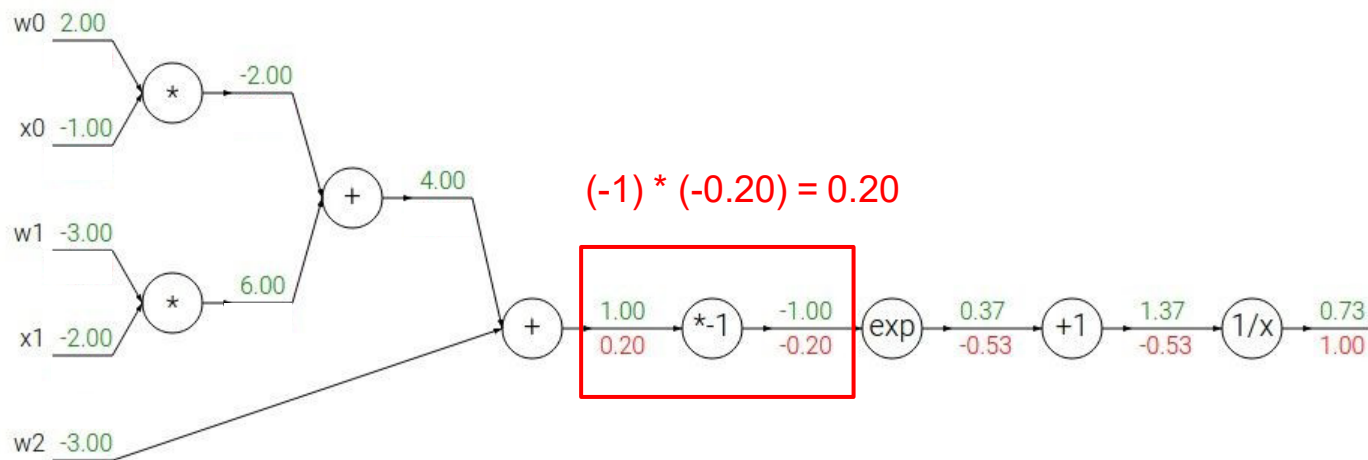
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



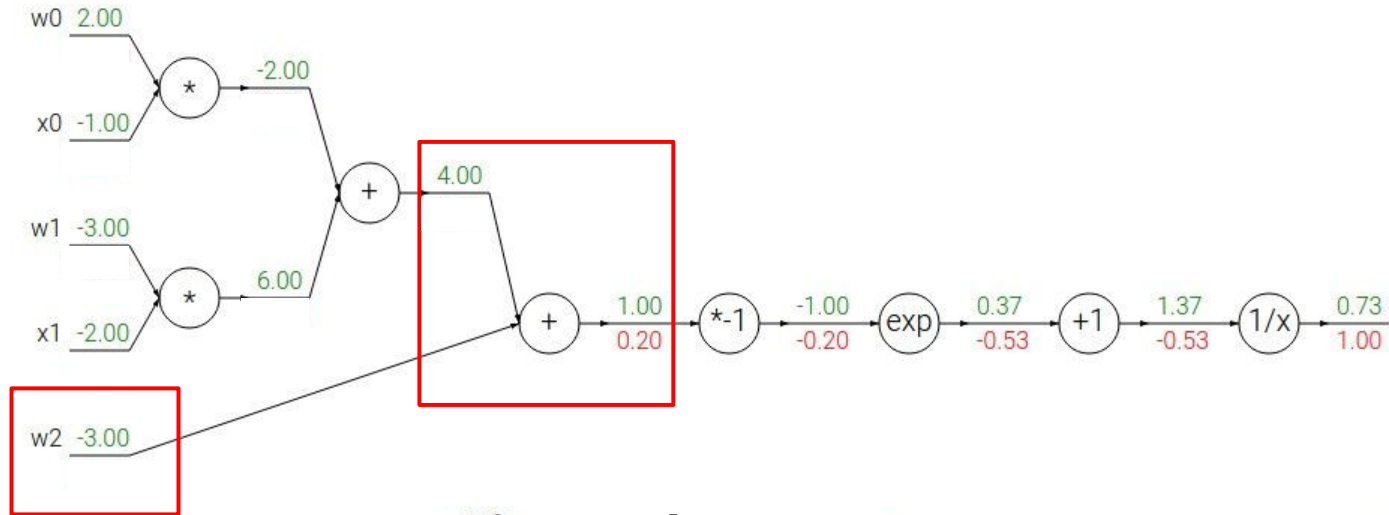
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

→

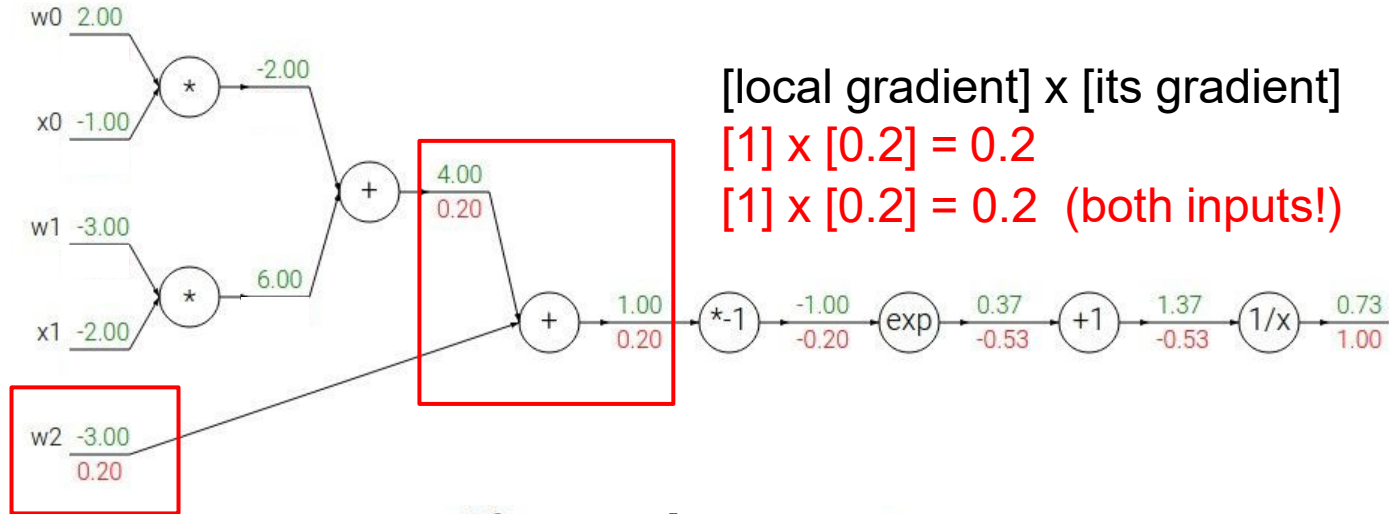
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

→

$$\frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



[local gradient] x [its gradient]

$$[1] \times [0.2] = 0.2$$

$$[1] \times [0.2] = 0.2 \text{ (both inputs!)}$$

$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

→

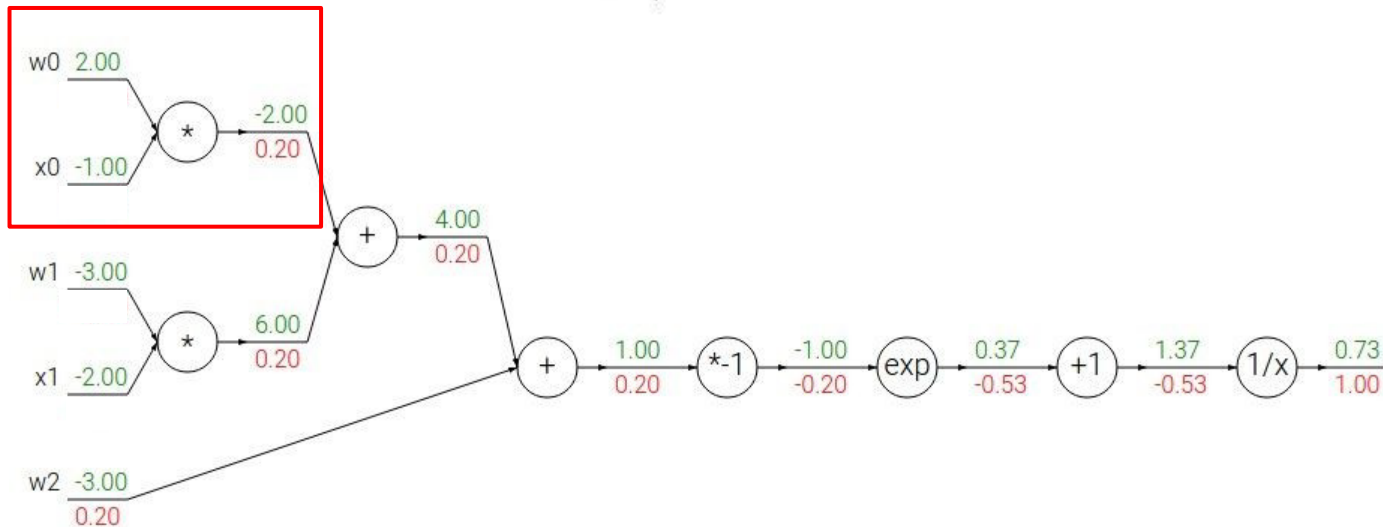
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

→

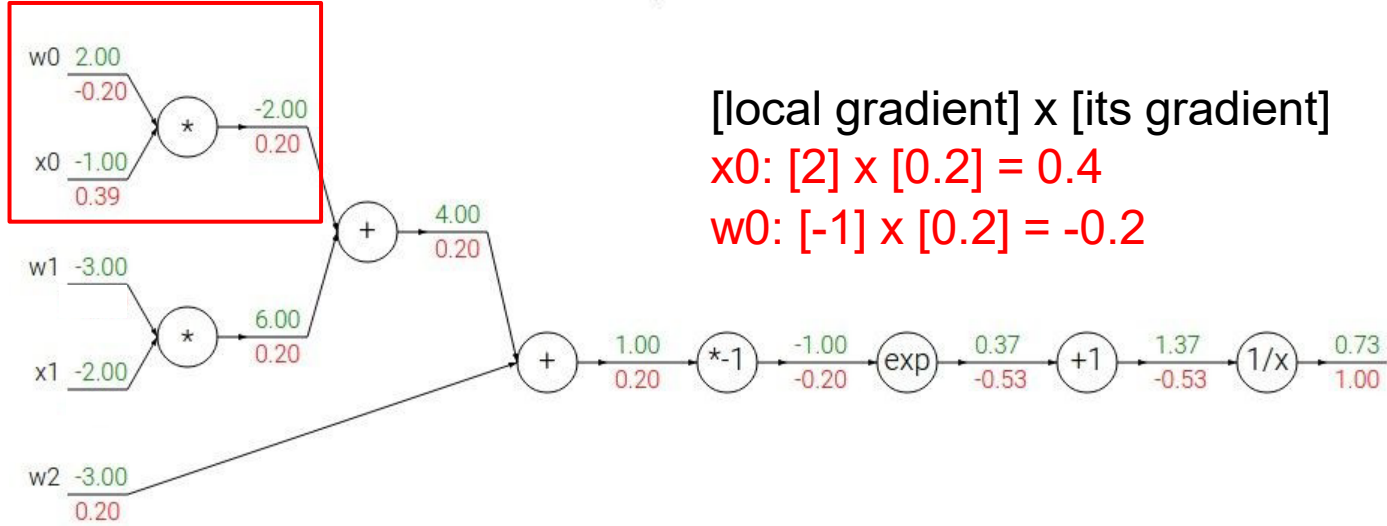
$$\frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



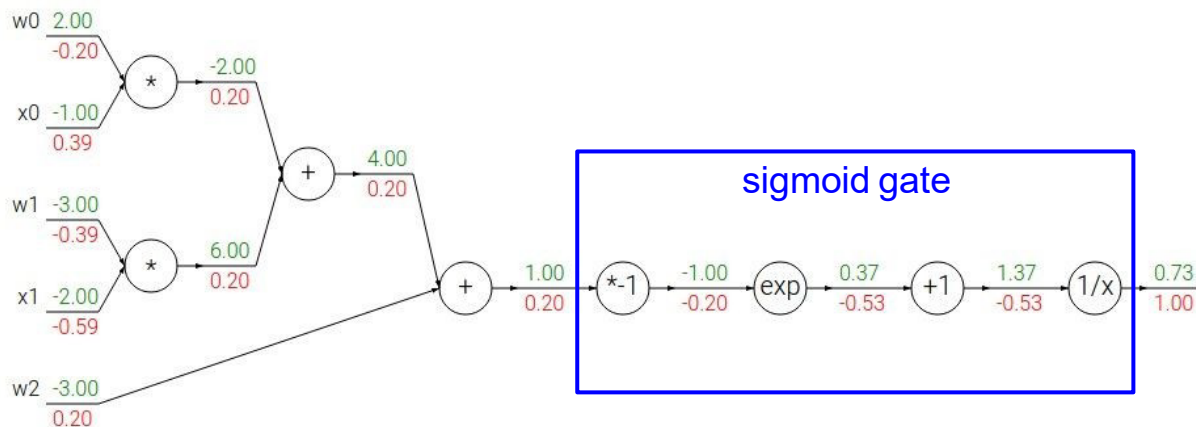
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

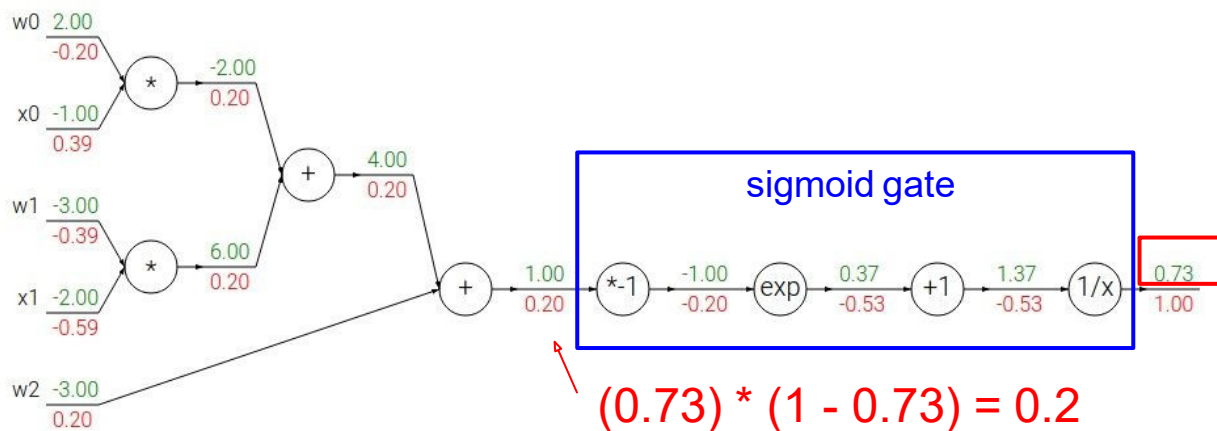


$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

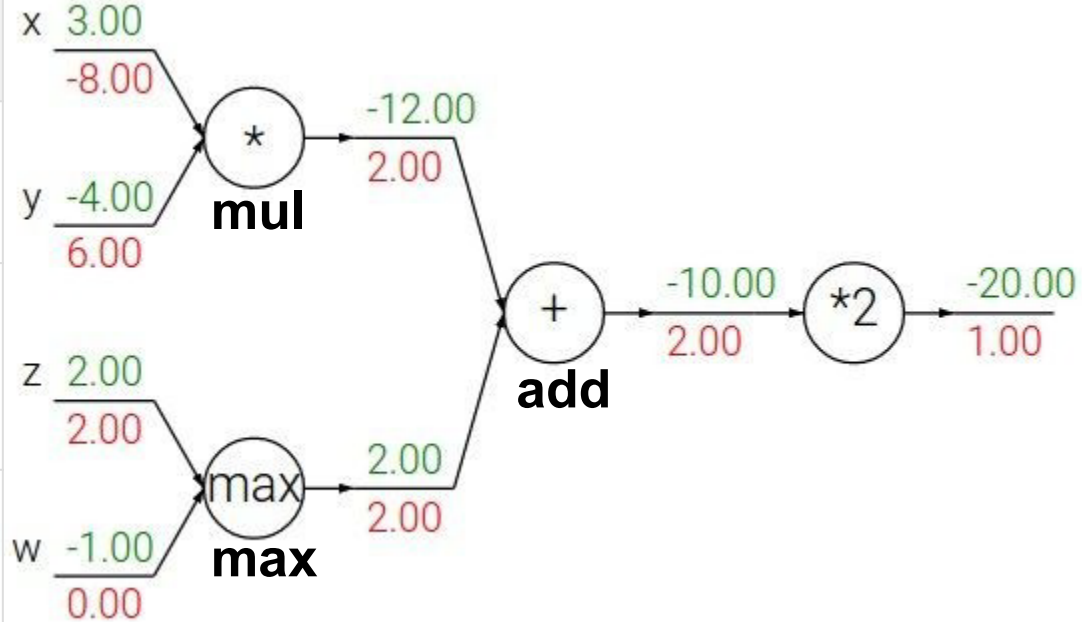
sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

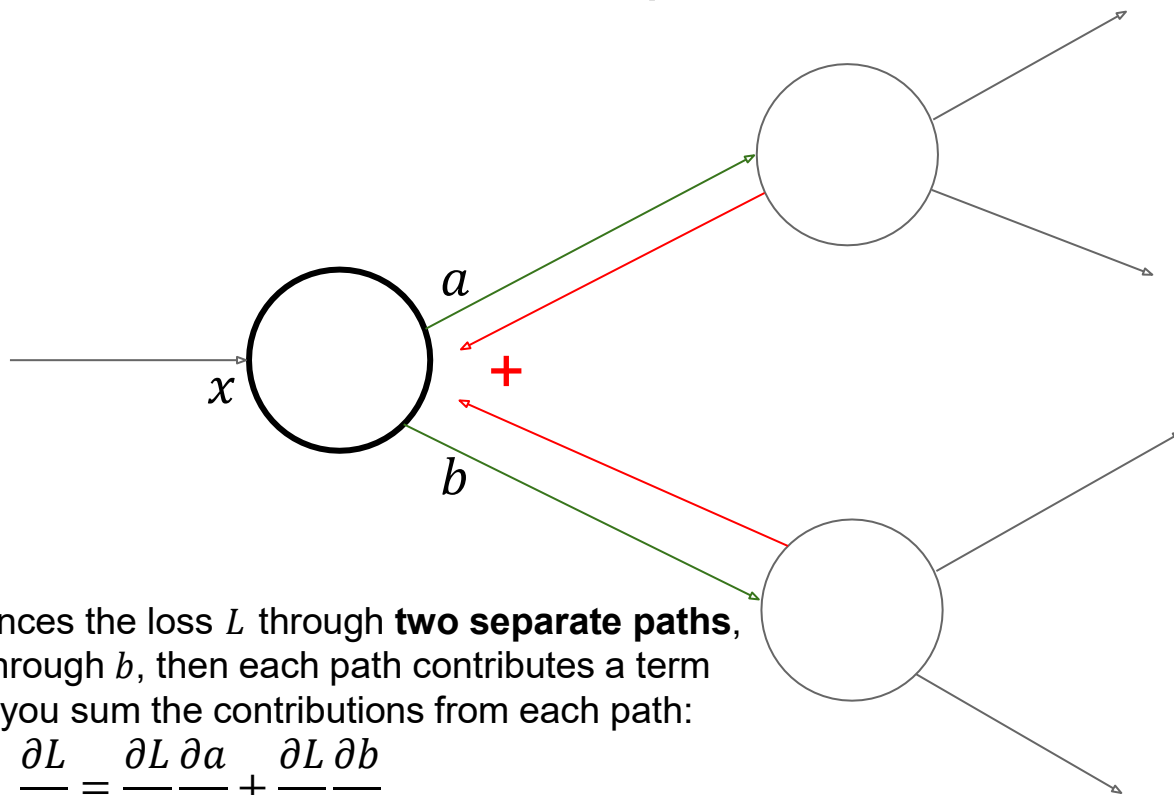


Patterns in backward flow

Gate	Behavior	Local gradient	Backward behavior
add	distributor	1 to every input	copies upstream gradient to all inputs unchanged
max	router	1 to the largest, 0 to the rest	routes the whole gradient to the larger input (winner) only
mul	switcher	the <i>other</i> input's value	each input gets upstream gradient $\times$ the other input



Gradients add up at branches



If a variable x influences the loss L through **two separate paths**, say through a and through b , then each path contributes a term to the gradient, and you sum the contributions from each path:

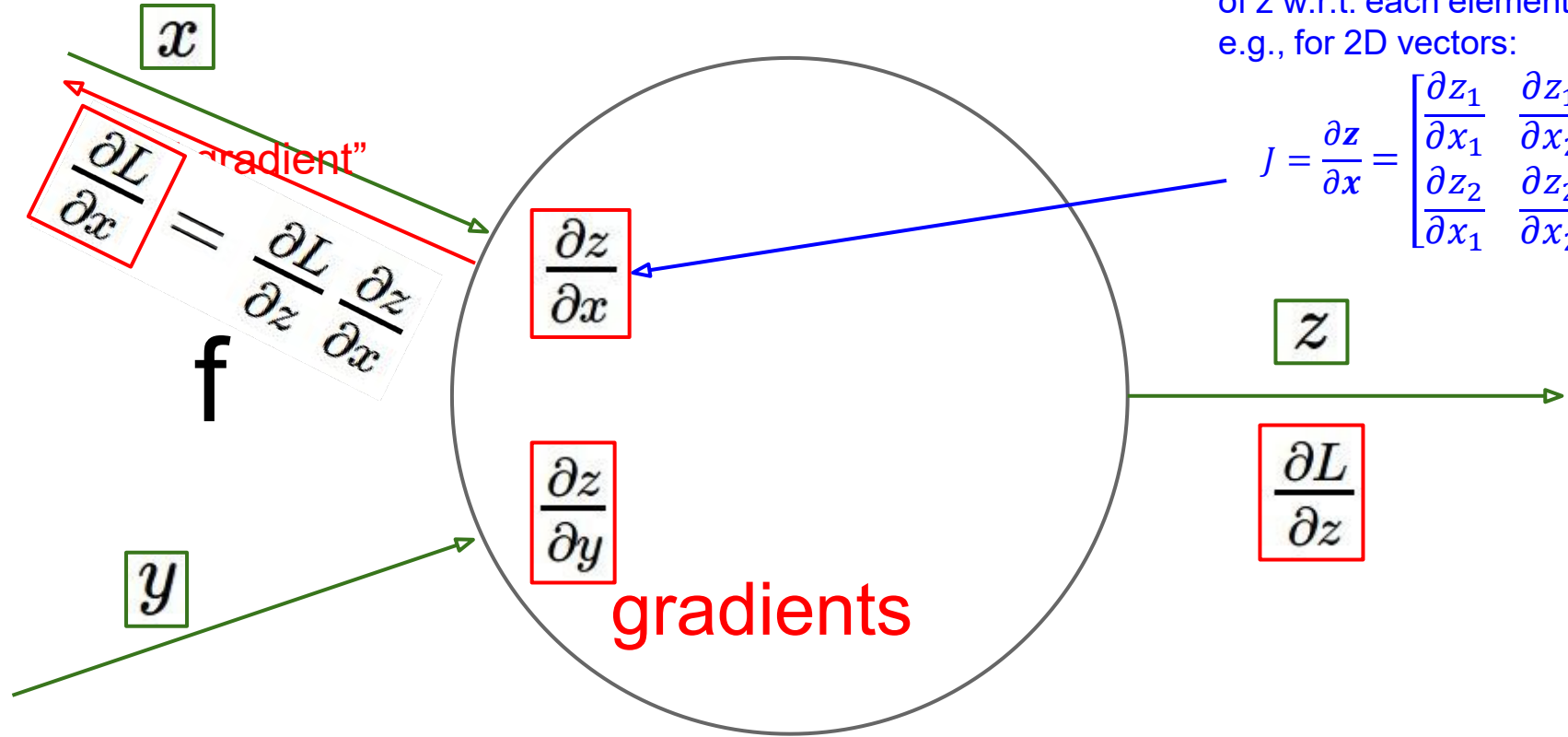
$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial L}{\partial b} \frac{\partial b}{\partial x}$$

(The total effect of nudging x is the **sum** of its effects along all paths.)

Gradients for vectorized code (x,y,z are now vectors)

Jacobian matrix:
derivative of each element
of z w.r.t. each element of x ,
e.g., for 2D vectors:

$$J = \frac{\partial z}{\partial x} = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \frac{\partial z_1}{\partial x_2} \\ \frac{\partial z_2}{\partial x_1} & \frac{\partial z_2}{\partial x_2} \end{bmatrix}$$



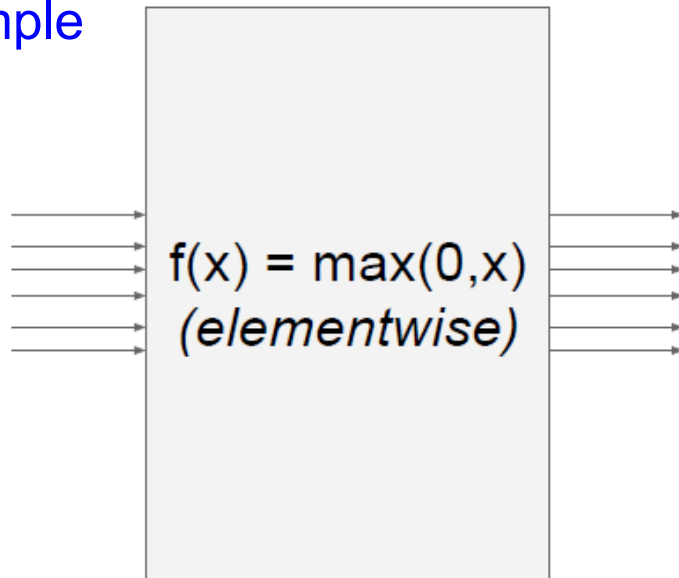
Vectorized operations

$$\frac{\partial L}{\partial x} = \boxed{\frac{\partial f}{\partial x}} \frac{\partial L}{\partial f}$$

Jacobian matrix

For one training sample

4096-d
input vector



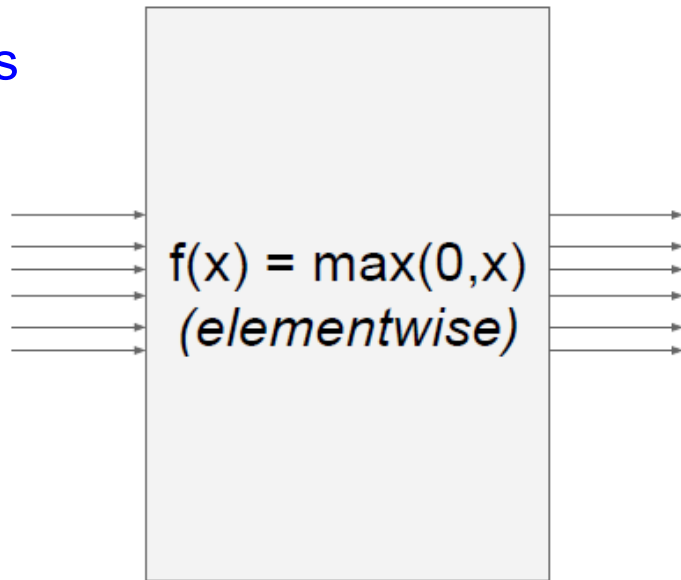
4096-d
output vector

Size of Jacobian
matrix:
[4096 x 4096]

Vectorized operations

In practice we process
one minibatch (e.g.
100) training samples
at once

100 4096-d
input vectors



Size of Jacobian
matrix:
[409,600 x 409,600]

$$\frac{\partial L}{\partial x} = \boxed{\frac{\partial f}{\partial x}} \frac{\partial L}{\partial f}$$

Jacobian matrix

100 4096-d
output vectors

Optimizers

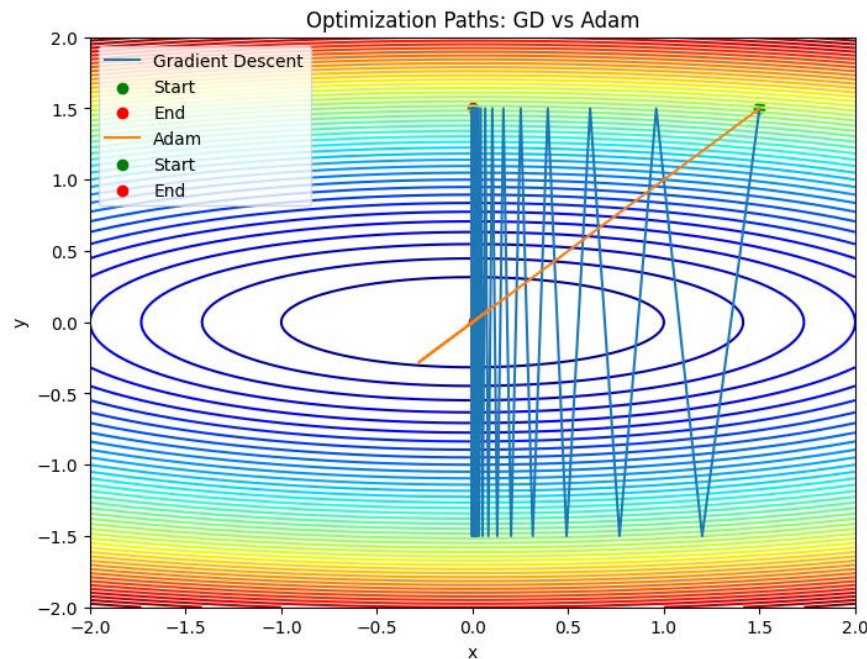
Stochastic/Mini-batch GD: *Speed improvement*

- Performs backprop on *one* training sample, or a *mini-batch* of samples instead of all samples
- Reduces the number of expensive backward passes
- Helps with getting out of local minima (due to higher gradient noise)

Optimizers

- Gradient Descent
- RMSProp
- Adaptive Momentum Estimation (**Adam**)

Figure shows GD struggles with oscillations in narrow valleys, leading to slower convergence, while Adam smoothly adjusts its path, leveraging adaptive learning rates and momentum to converge efficiently.



<https://www.vizuaranewsletter.com/p/adam-the-most-famous-ml-optimizer>

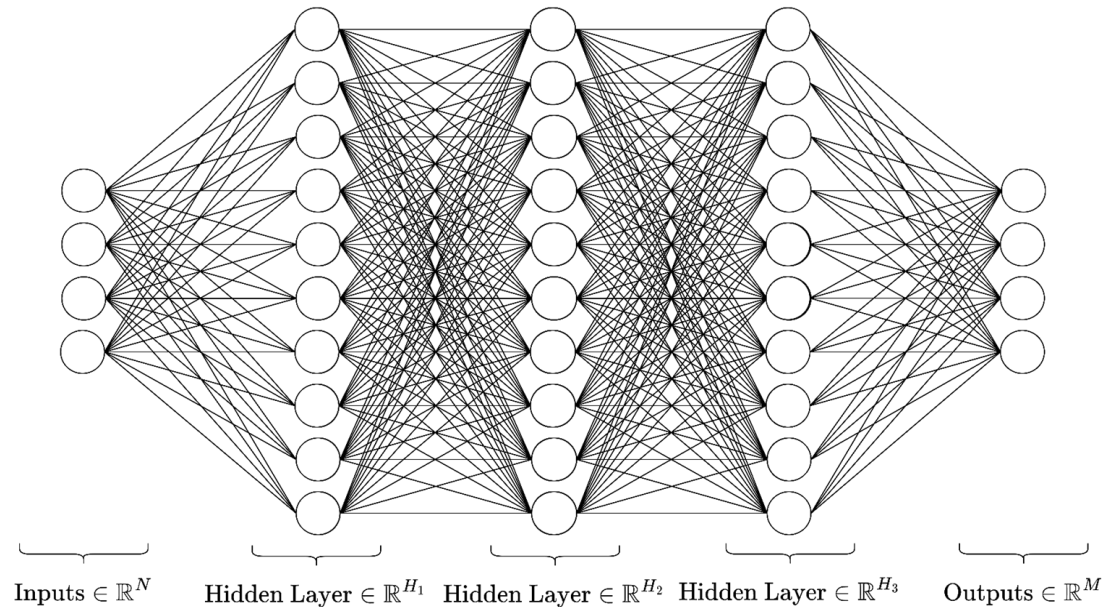
Types of Neural Networks

Multi-layer Perceptrons

Useful for **static** input-output relations

More hidden layers → **better approximation** of more complicated functions

Quick to design and implement

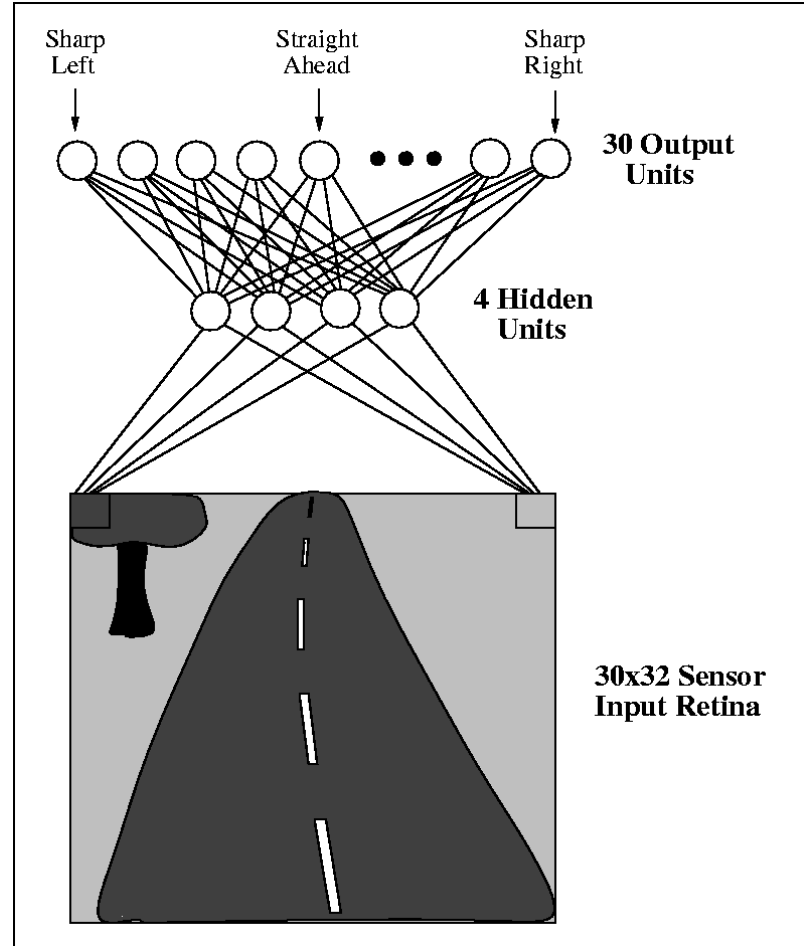


ALVINN

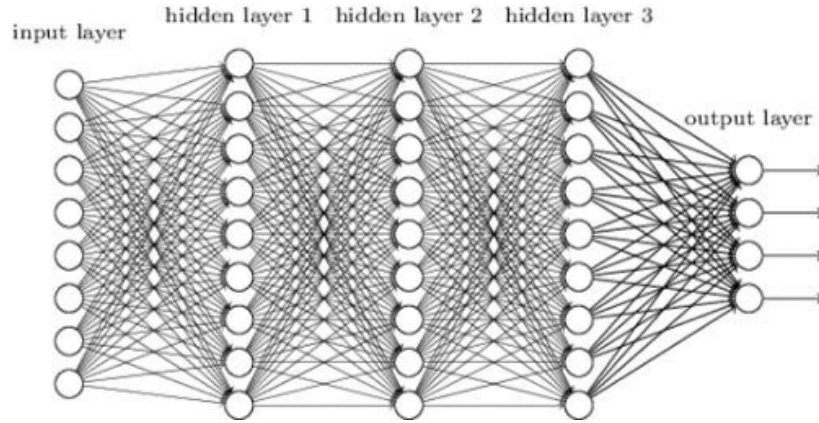
CMU's **Autonomous Land Vehicle In a Neural Network (ALVINN)** (1988-89) can drive 70mph on highways. ALVINN's architecture is a MLP with a single hidden layer.

Input layer is a 30x32 two dimensional "retina" which receives input from camera.

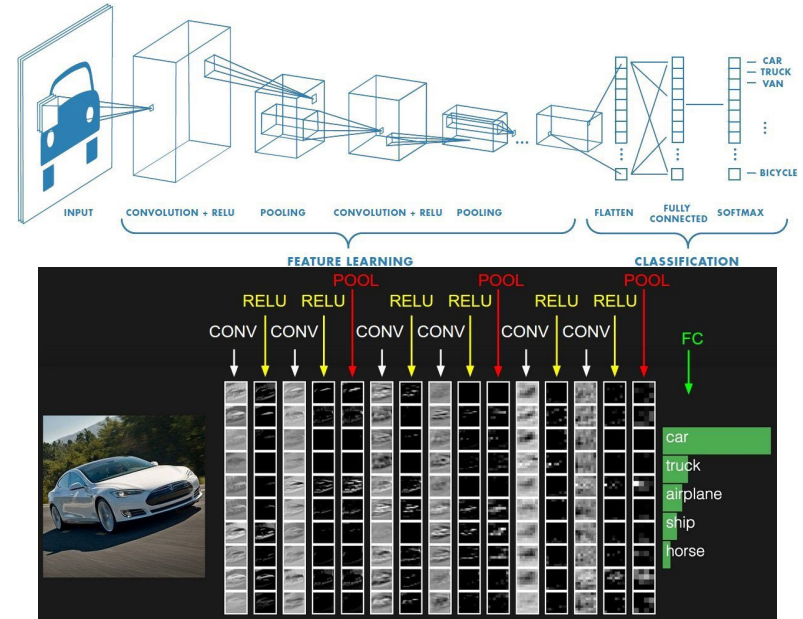
Output layer (30 neurons) encodes the direction the vehicle should travel to keep it on the road (as a **30-class classification task**)



MLP vs. Convolutional Neural Network (CNN)



In an **MLP**, all layers are Fully-Connected
Cannot alter input image size
No translation invariance
No. weight params can grow very large, prone to overfitting



In a **CNN**, only the last few layers are FC
Each CONV layer extracts increasingly higher level features, followed by FC layers for classification
Translation invariance (object can be anywhere in the image)
Fewer params than MLP

Convolutional Neural Network (CNN)

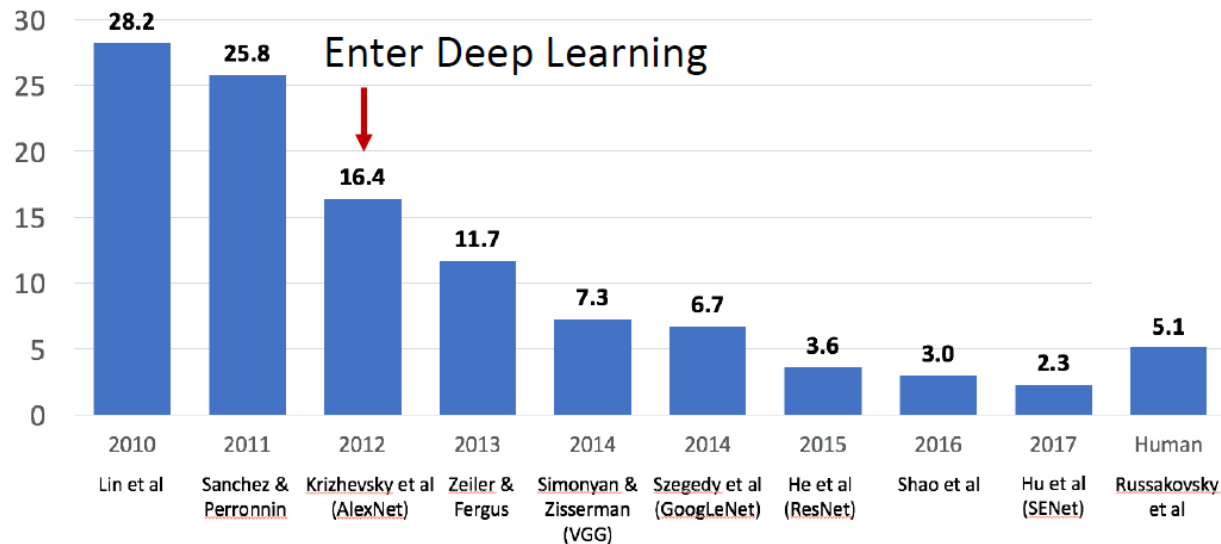
ImageNet Large Scale Visual Recognition Challenge:
1000 object classes, 1.4 M labeled images.

Revolution of Depth

AlexNet, 8 layers
(ILSVRC 2012)

VGG, 19 layers
(ILSVRC 2014)

ResNet, 152 layers
(ILSVRC 2015)



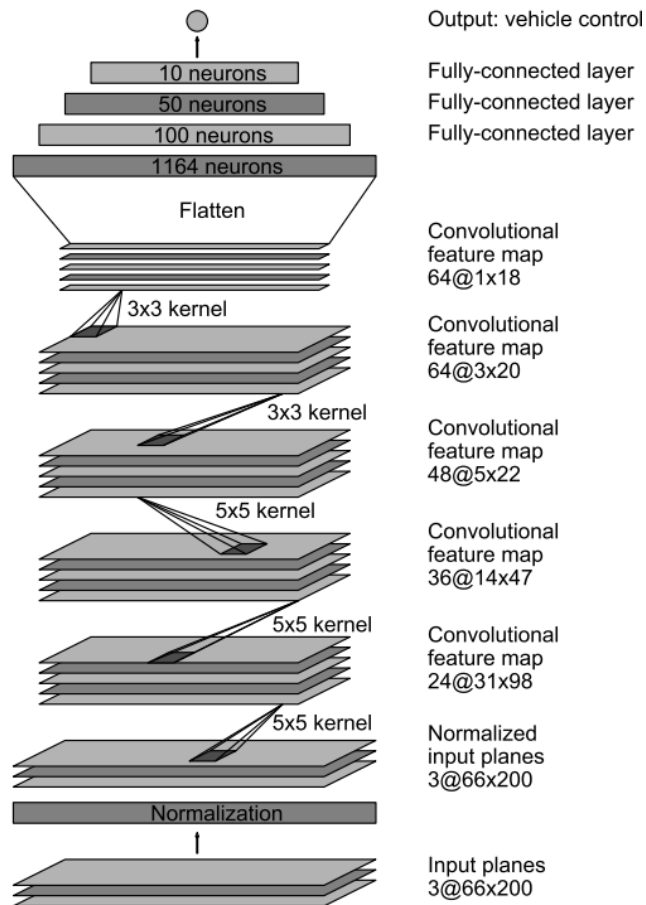
Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". CVPR 2016.



NVIDIA PilotNet

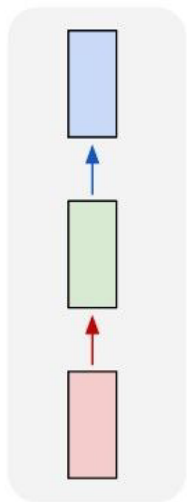
NVIDIA PilotNet (2016) is an **end-to-end autonomous driving** system that uses a CNN to directly predict steering commands from camera images (as a **regression task**)

It demonstrated feasibility of learning driving behavior directly from data, paving the way for modern AI-based self-driving technologies.



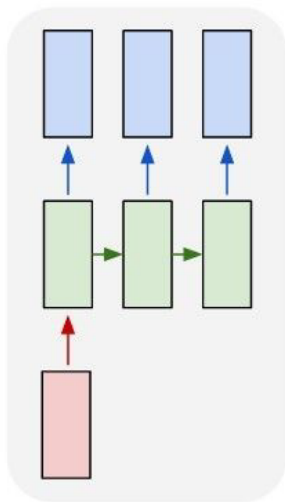
Recurrent Neural Network

one to one



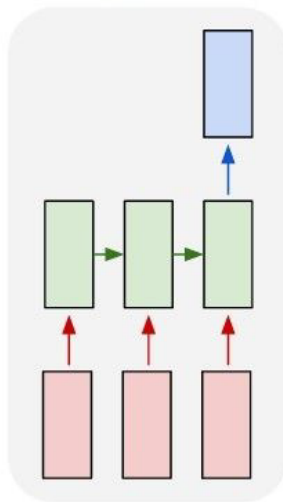
Regular
Feedforward NN

one to many



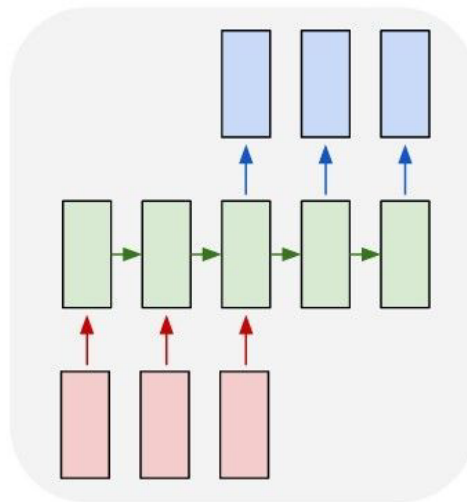
e.g. Image Captioning
image -> sequence of
words

many to one



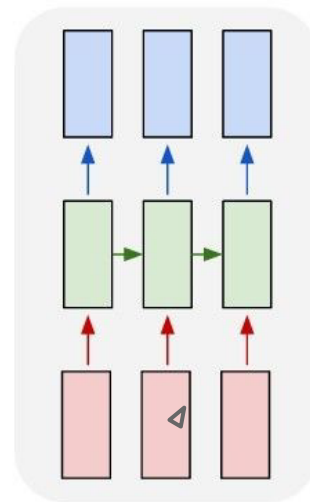
e.g. Sentiment
Classification
sequence of words ->
sentiment

many to many



e.g. Machine Translation
seq of words -> seq of words

many to many

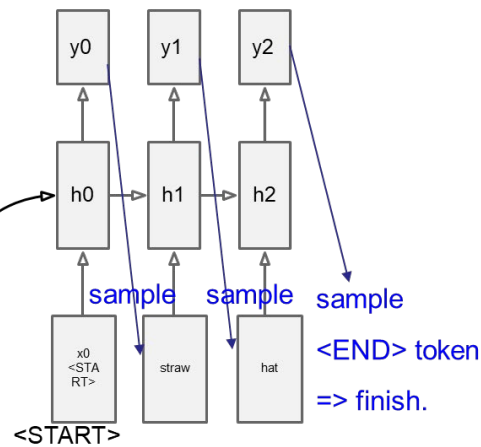
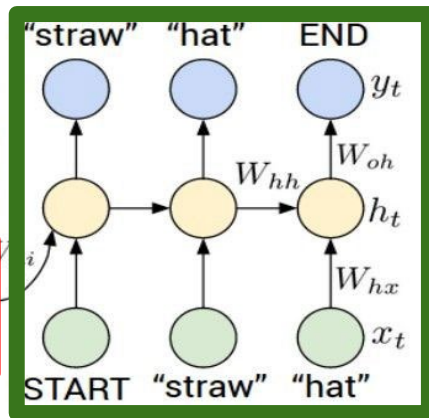
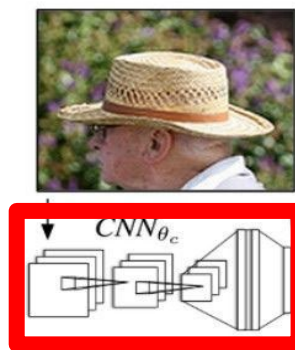


e.g., video
classification
on frame level

Image Captioning: CNN + RNN

CNN processes the input image and generates a feature vector as input to **RNN**

Last 2 layers of the CNN for classification (FC-1000 and SoftMax) are not used, since we only need the extracted features from the layer FC-4096



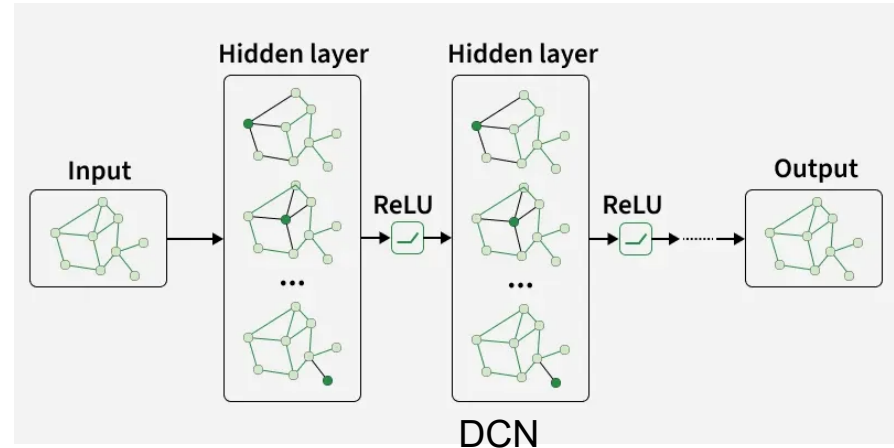
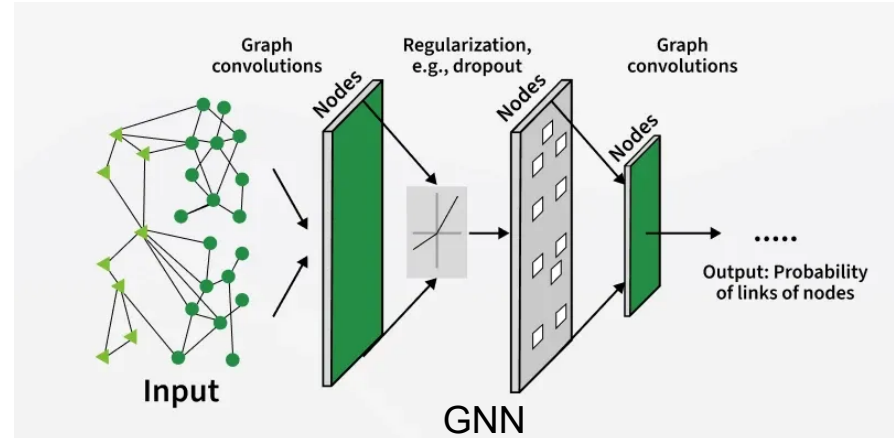
Graph Neural Network

GNNs operate on graphs made of nodes and edges.

Information is passed between connected nodes (neighbors) through message-passing.

Graph Convolutional Networks (GCNs) extend the idea of convolution from grid data to graphs.

Useful for tasks like social network analysis, molecule prediction, and recommendation systems.



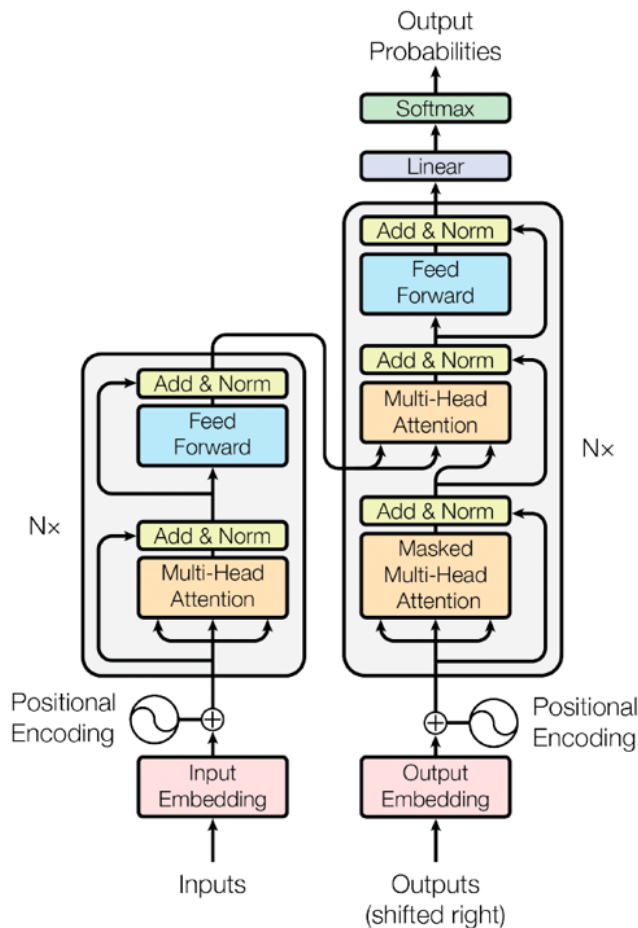
Transformer

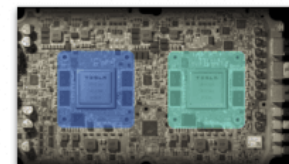
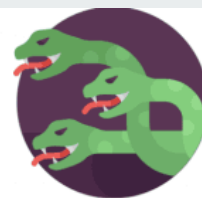
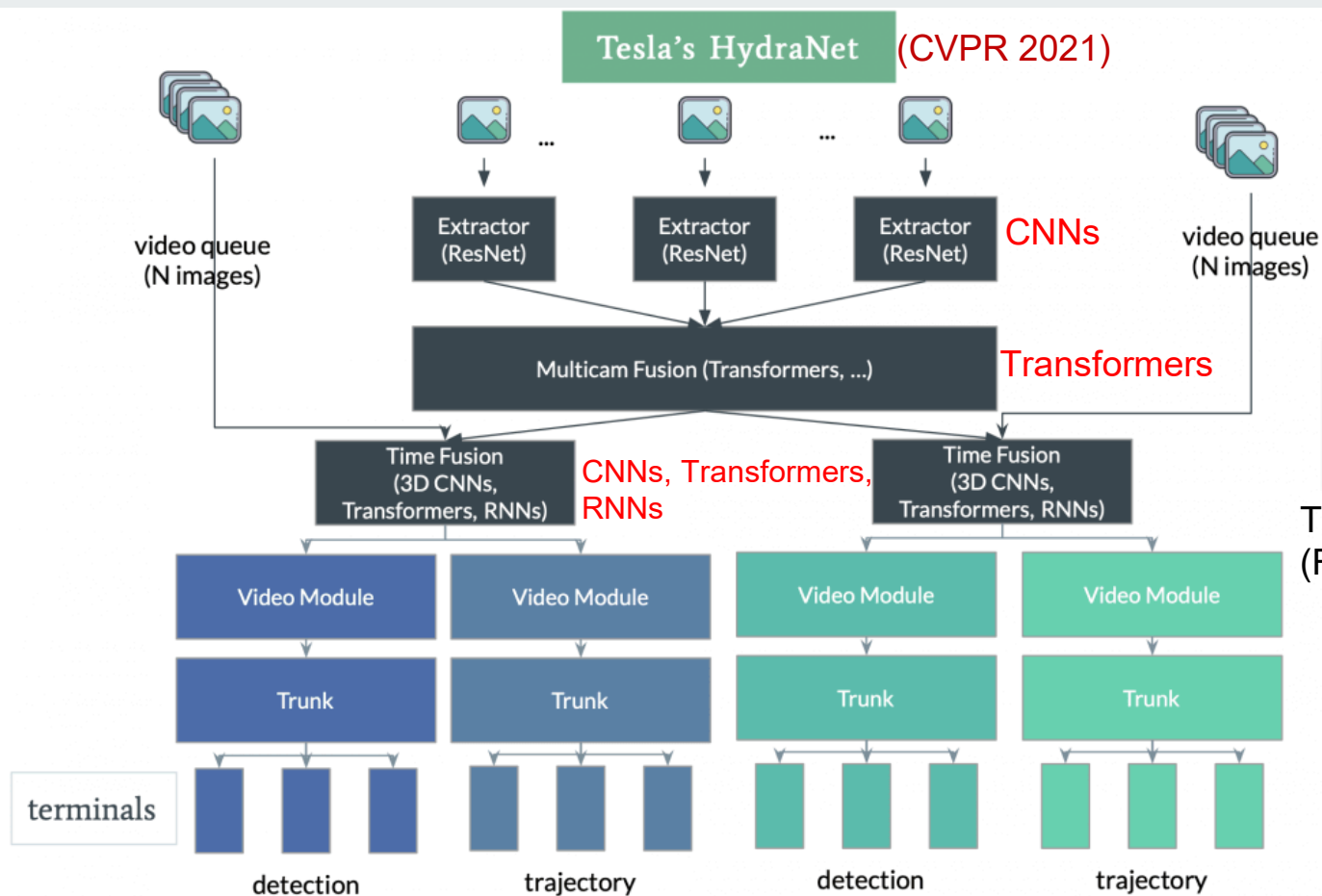
Uses attention mechanisms to capture relationships between inputs

Processes entire sequences at once instead of step by step

Improves performance on tasks involving context and dependencies

Basis for modern Foundation Models (LLMs/ VLMs...)





Tesla's Full Self-Driving (FSD) Chip

Training tips



Overfitting & Underfitting

The real troublemakers in ML in general!

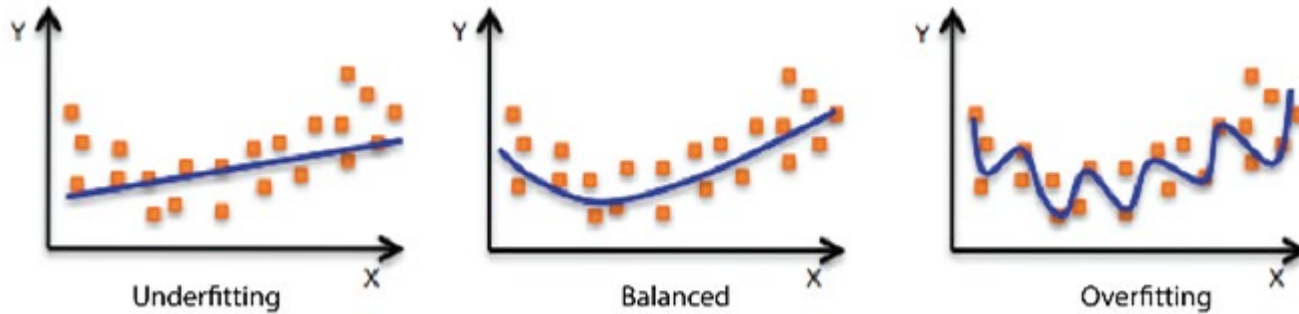
Underfitting: When the model fits the training data not well enough

- Empirical risk is high, actual risk is high
- Training loss is high, testing loss is not optimal

Overfitting: When the model fits the training data *too* closely (incl. noise)

- Empirical risk is low, actual risk is high
- Training loss is low, testing loss is not optimal
- e.g. An D -degree polynomial can fit $D-1$ training points with *zero* error

Overfitting & Underfitting



More complex models (e.g. more layers, neurons per layer) -> higher likelihood of overfitting



Validation

Split your training set into two!

- New train set
- Unseen-by-the-model “validation” set

Train Set

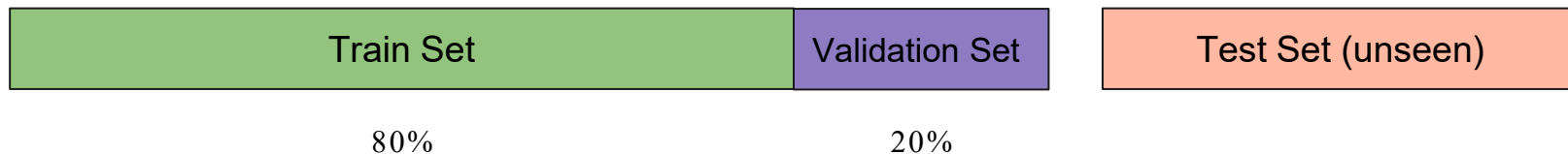
Test Set (unseen)



Validation

Split your training set into two!

- New train set
- Unseen-by-the-model “validation” set
- e.g. 80-20 split (Note: split ratio depends on the model, task and data)

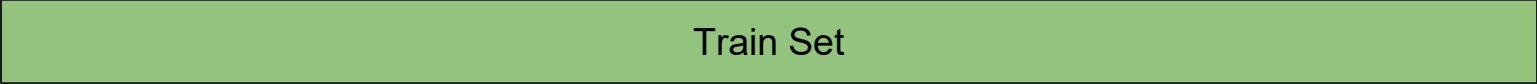




***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models



Train Set



***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models

e.g. $k=5$ (5-fold cross-validation)

Train Set	Train Set	Train Set	Train Set	Train Set
-----------	-----------	-----------	-----------	-----------

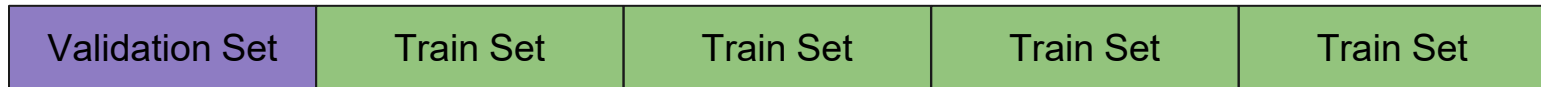


***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models

e.g. $k=5$ (5-fold cross-validation)



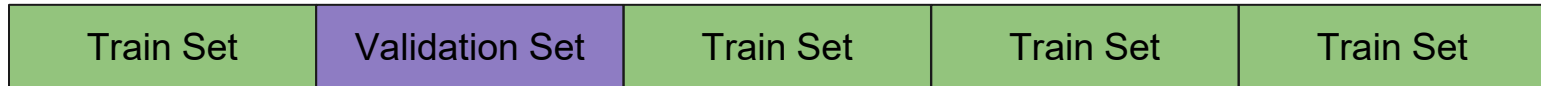


***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models

e.g. $k=5$ (5-fold cross-validation)





***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models

e.g. $k=5$ (5-fold cross-validation)



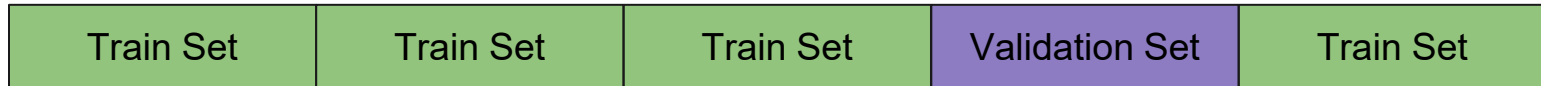


***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models

e.g. $k=5$ (5-fold cross-validation)



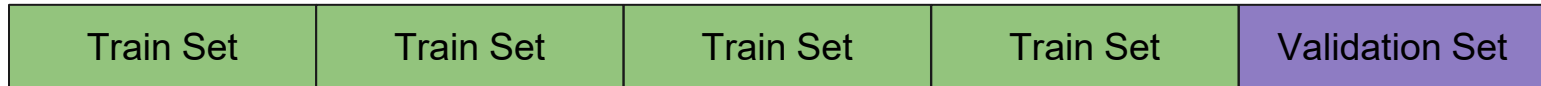


***k*-fold Cross-validation**

Split training set into k -segments, iteratively train and validate with each segment.

- Accounts for irregularities in training set
- “Gold standard” for evaluating generality of neural network models

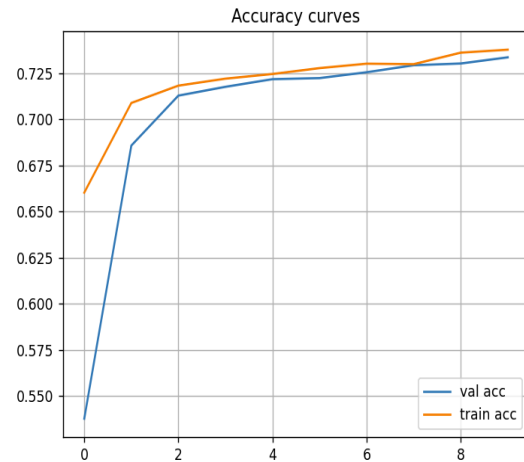
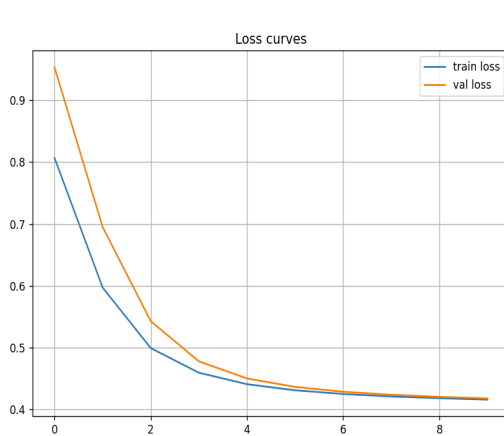
e.g. $k=5$ (5-fold cross-validation)



Result = average over all validation passes

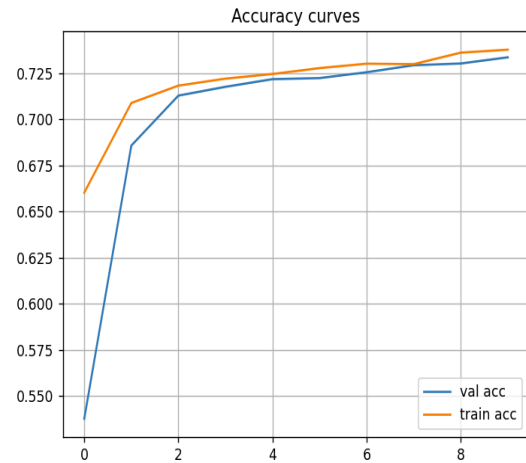
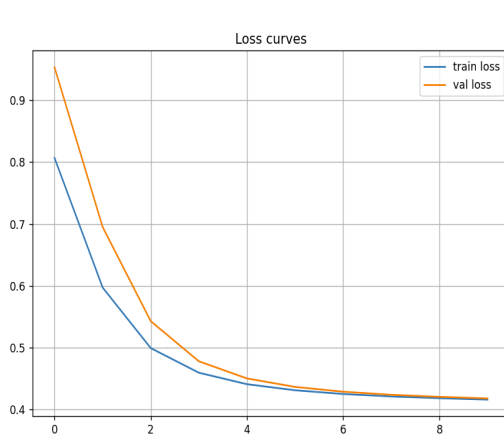
Training curves

Important to plot!



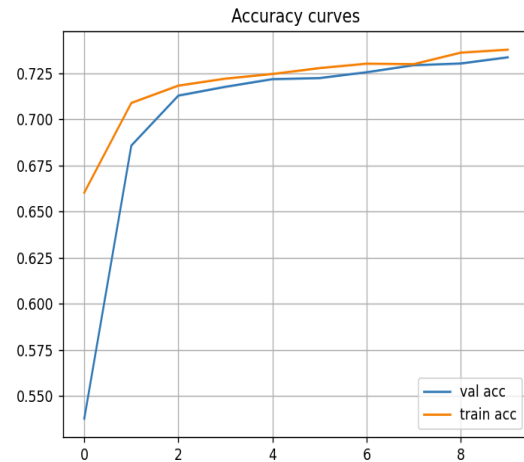
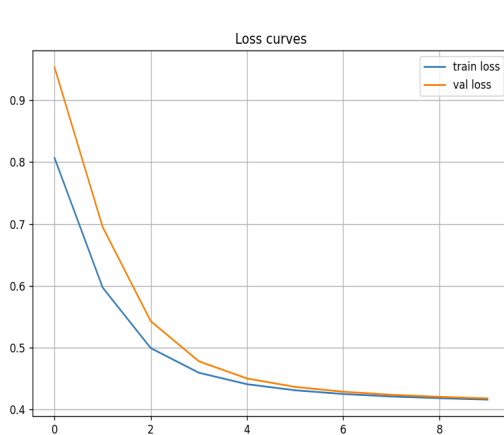
Training curves

Important to plot!!!!



Training curves

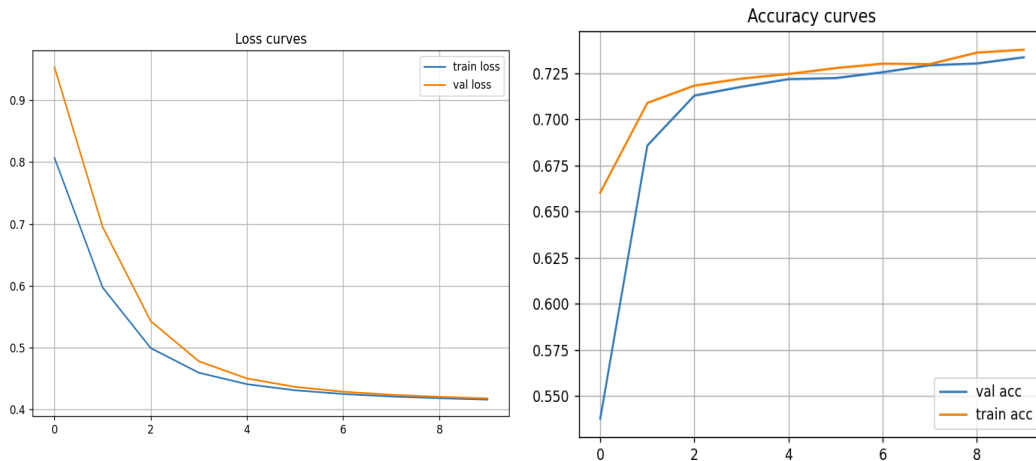
Important to plot!!!!



Shows if and how fast your model is learning on task-relevant metrics

- e.g. loss, accuracy, AUC, F1 score
- Plot scores over training epochs

Training curves



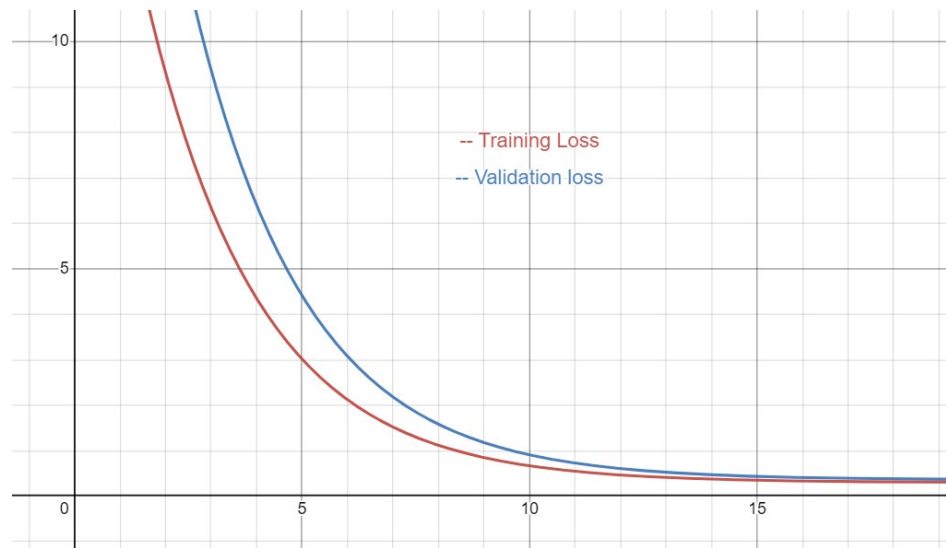
Shows if and how fast your model is learning on task-relevant metrics

- e.g. loss, accuracy, AUC, F1 score
- Plot scores over training epochs

May indicate potential over and underfitting

Reading training curves

If



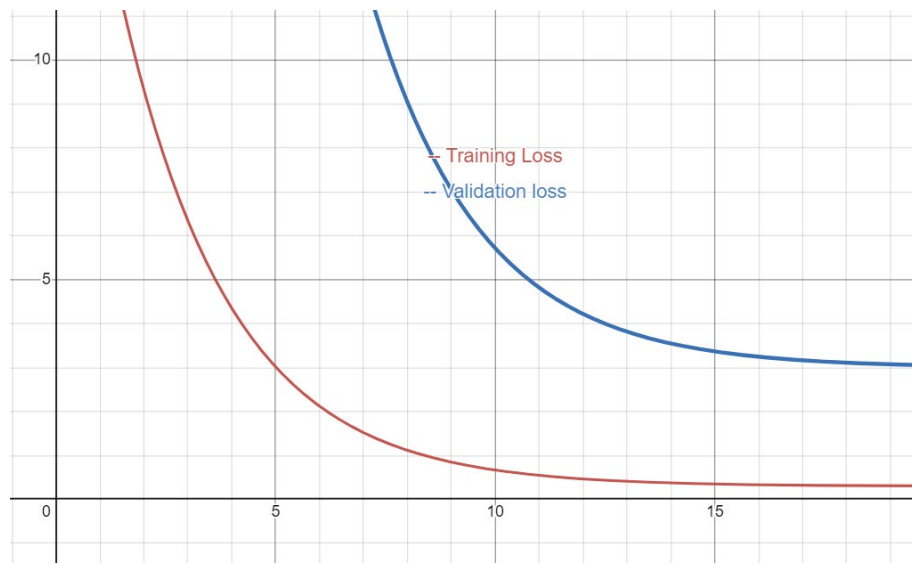
validation loss > training loss

then often the model is *good!*

Low loss == Better

Reading training curves

If



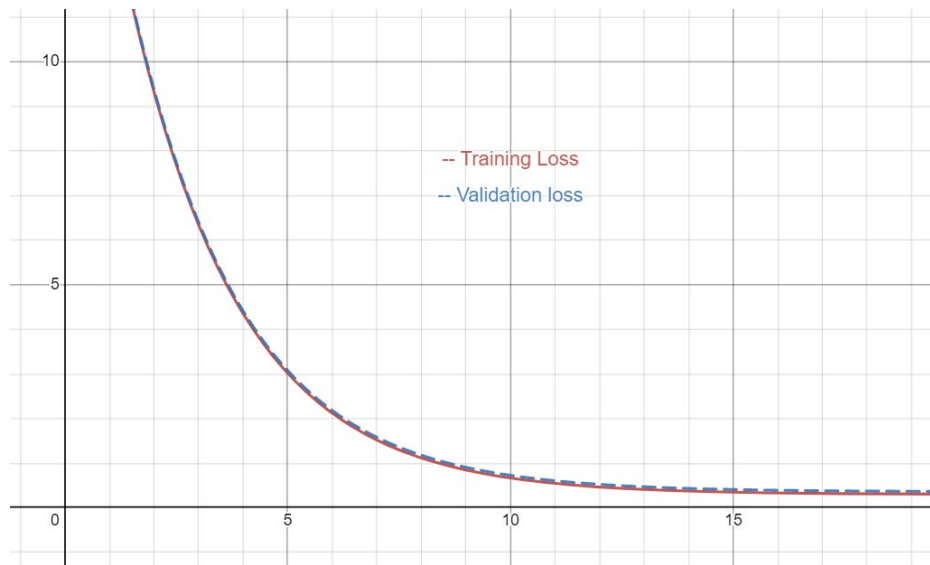
validation loss $\gg$ training loss

Low loss == Better

then often the model is *overfitting*

Reading training curves

If



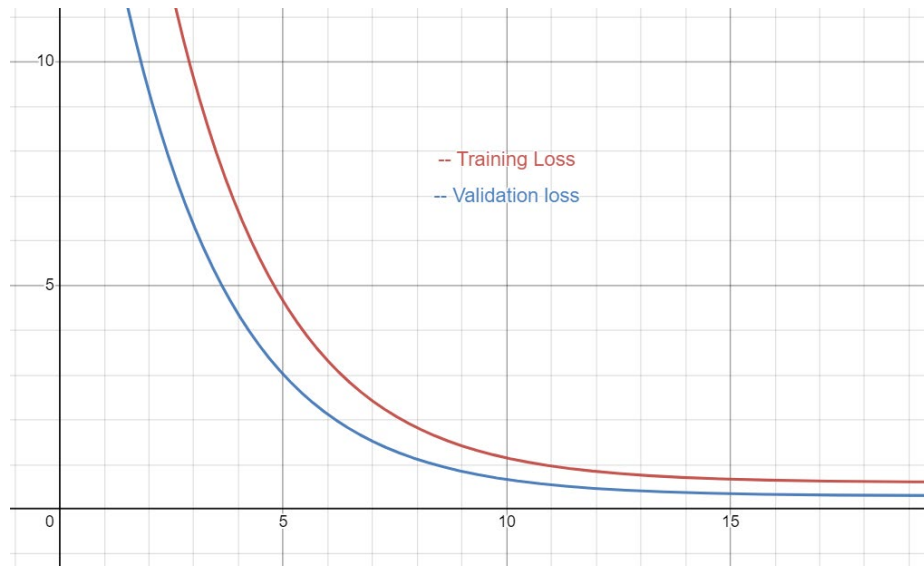
validation loss $\sim$ training loss

Low loss == Better

then often the model is *underfitting*

Reading training curves

If



validation loss < training loss

then something is *very* wrong!

Low loss == Better



Weight Regularization

- L1/L2 Regularization helps make weights sparse (L1), or have small magnitude (L2):

$$\lambda \cdot \sum_{i=1}^d |w_i| \qquad \lambda \cdot \sum_{i=1}^d w_i^2$$

- Dropout (on when training, off when testing/deploying)
- Early stopping

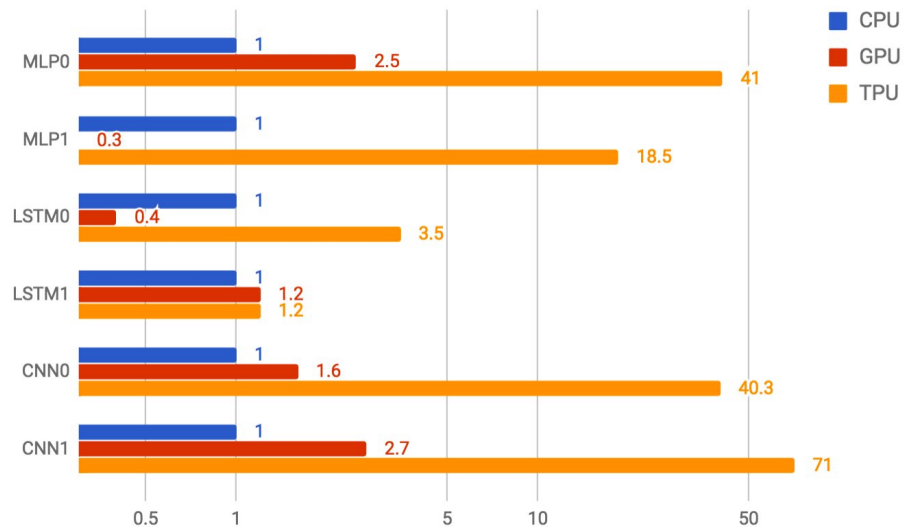
Parallelization: Speeding up NNs

Main math operation in NNs:

- Matrix-vector multiplications
- Element-wise nonlinear activation functions

Parallelization can be used to **massively speed up** learning and deployment!

- Graphics processing units (GPUs)
- Neural Processing Units (NPU), typically ASICs, e.g., TPU (Google)
- FPGAs



<https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu>



Where NNs thrive

- > Statistical/ correlation inference
- > There exists a lot of good quality (labelled) training data
- > Parallelizable training and deployment

Limits of NNs

- > Very data hungry - “*Garbage in, garbage out*”
- > Expensive to train (depending on size)
- > Non-extensible and specialized to a range and task
 - Add one more neuron → needs fine-tuning
 - Undefined behaviour on out-of-distribution examples

Frontiers

Shallow NNs → Deep NNs with hundreds of layers, billions of weights
CNNs → Transformers → Foundation Models (LLMs/ VLMs/ VLAs)
Supervised Learning → Self-Supervised Learning, Reinforcement
Learning, RL with Human Feedback...

